

Markov Chain Monte Carlo Diagnostics

Michael Betancourt

2023-01-01

Table of contents

1	Extraction	2
2	Hamiltonian Monte Carlo Diagnostics	4
2.1	Check Hamiltonian Monte Carlo Diagnostics	4
2.2	Integrator Inverse Metric Elements	8
2.3	Integrator Step Sizes	10
2.4	Numerical Trajectory Lengths	10
2.5	Average Proxy Acceptance Statistic	12
2.6	Divergence-Labeled Pairs Plot	13
3	Expectand Diagnostic Functions	19
3.1	xihat	19
3.2	Frozen Chains	24
3.3	Split Rhat	25
3.4	Integrated Autocorrelation Time	28
3.5	All Expectand Diagnostics	34
3.6	Empirical Autocorrelation Visualization	46
3.7	Chain-Separated Pairs Plot	48
4	Markov Chain Monte Carlo Estimation	49
5	Demonstration	62
	License	94
	Original Computing Environment	94

In this short note I will preview the new suite of Markov chain Monte Carlo analysis tools that I will be introducing more formally in upcoming writing. These tools largely focus on

diagnostics but there are also a few that cover Markov chain Monte Carlo estimation assuming a central limit theorem.

We'll start with diagnostics specific to Hamiltonian Monte Carlo then consider more generic diagnostics that consider each expectand of interest one at a time. Finally we'll look at a way to visualize one-dimensional pushforward distributions using Markov chain Monte Carlo to estimate bin probabilities.

Before any of that, however, we need to set up our graphics.

```
par(family="serif", las=1, bty="l", cex.axis=1, cex.lab=1, cex.main=1,
    xaxs="i", yaxs="i", mar = c(5, 5, 3, 5))

c_light <- c("#DCBCBC")
c_light_highlight <- c("#C79999")
c_mid <- c("#B97C7C")
c_mid_highlight <- c("#A25050")
c_dark <- c("#8F2727")
c_dark_highlight <- c("#7C0000")

c_light_teal <- c("#6B8E8E")
c_mid_teal <- c("#487575")
c_dark_teal <- c("#1D4F4F")
```

1 Extraction

The `extract` function in `RStan` parses the Markov chain output within a `StanFit` object into a usable format. Due to some unfortunate choices in early development, however, the function behaves a bit awkwardly.

By default it permutes the Markov chain iterations and then aggregates them together. This permutation strips the iterations of their autocorrelations, making it impossible to recover accurate estimates of the Markov chain Monte Carlo estimator error.

There is an optional argument that deactivates the permutation, but that also completely changes the output format. In particular it strips the expectands of their names, requiring that users access each expectand by the order in which they appear in the original Stan program.

Finally the `extract` function also ignores all of the Hamiltonian Monte Carlo diagnostic information emitted at each transition. Instead the `get_sampler_params` function recovers this information, albeit it yet another output format.

To facilitate the analysis of Stan output I've included my own custom extract functions that format the Markov chain Monte Carlo output into named lists, with one named element for each expectand or Hamiltonian Monte Carlo diagnostic. The elements themselves are two-dimensional arrays with the first index denoting the individual Markov chains and the second index denoting the iterations within an individual Markov chain.

```
# Extract unpermuted expectand values from a StanFit object and format
# them for convenient access. Removes the auxiliary `lp__` variable.
# @param stan_fit A StanFit object
# @return A named list of two-dimensional arrays for each expectand in
#         the StanFit object. The first dimension of each element
#         indexes the Markov chains and the second dimension indexes the
#         sequential states within each Markov chain.
extract_expectand_vals <- function(stan_fit) {
  nom_params <- rstan::extract(stan_fit, permuted=FALSE)
  N <- dim(nom_params)[3] - 1
  params <- lapply(1:N, function(n) t(nom_params[,n]))
  names(params) <- names(stan_fit)[1:N]
  (params)
}
```

```
# Extract Hamiltonian Monte Carlo diagnostics values from a StanFit
# object and format them for convenient access.
# @param stan_fit A StanFit object
# @return A named list of two-dimensional arrays for each expectand in
#         the StanFit object. The first dimension of each element
#         indexes the Markov chains and the second dimension indexes the
#         sequential states within each Markov chain.
extract_hmc_diagnostics <- function(stan_fit) {
  diagnostic_names <- c('divergent__', 'treedepth__', 'n_leapfrog__',
                        'stepsize__', 'energy__', 'accept_stat__')

  nom_params <- get_sampler_params(stan_fit, inc_warmup=FALSE)
  C <- length(nom_params)
  params <- lapply(diagnostic_names,
                  function(name) t(sapply(1:C, function(c)
                                          nom_params[c][[1]][,name])))
  names(params) <- diagnostic_names
  (params)
}
```

If users are able to modify these functions to accept the output from other interfaces to Stan and return the same output format then all of the following functions will be immediately

available. That is all except for the `plot_inv_metric` function which does require a separate RStan-specific function for extracting adaptation information.

2 Hamiltonian Monte Carlo Diagnostics

Hamiltonian Monte Carlo introduces a suite of powerful diagnostics that can identify obstructions to Markov chain Monte Carlo central limit theorems. These diagnostics are not only extremely sensitive but also probe the behavior of the entire Markov chain state instead of the projections of that state through single expectands.

2.1 Check Hamiltonian Monte Carlo Diagnostics

All of our diagnostics are assembled in this single `check_all_hmc_diagnostics` function.

The first diagnostic looks for unstable numerical Hamiltonian trajectories, or divergences. These unstable trajectories are known to obstruct typical central limit theorem conditions. Divergences arise when the target distribution is compressed into a narrow region; this forces the Hamiltonian dynamics to accelerate which makes them more difficult to accurately simulate.

Increasing `adapt_delta` will on average result in a less aggressive step size optimization that in some cases may improve the stability of the numerical integration but at the cost of longer, and hence more expensive, numerical Hamiltonian trajectories. In most cases, however, the only productive way to avoid divergences is to reparameterize the ambient space to decompress these pinches in the target distribution.

Stan's Hamiltonian Monte Carlo sampler expands the length of the numerical Hamiltonian trajectories dynamically to maximize the efficiency of the exploration. That length, however, is capped at $2^{\text{max_treedepth}}$ steps to prevent trajectories from growing without bound.

When numerical Hamiltonian trajectories are long but finite this truncation will limit the computational efficiency. Increasing `max_treedepth` allow the trajectories to expand further. While the resulting trajectories will be more expensive that added cost will be more than made up for by increased computational efficiency.

The energy fraction of missing information, or E-FMI, quantifies how well the Hamiltonian dynamics are able to explore the target distribution. If the E-FMI is too small then even the exact Hamiltonian trajectories will be limited to confined regions of the ambient space and full exploration will be possible only with the momenta resampling between trajectories. In this case the Markov chain exploration devolves into less efficient, diffusive behavior where Markov chain Monte Carlo estimation is fragile at best.

This confinement is caused by certain geometries in the target distribution, most commonly a funnel geometry where some subset of parameters shrink together as another parameter ranges

across its typical values. The only way to avoid these problems is to identify the problematic geometry and then find a reparameterization of the ambient space that transforms the geometry into something more pleasant.

Finally the average proxy accept statistic is a summary for Stan's step size adaptation. During warmup the integrator step size is dynamically tuned until this statistic achieves the target value which defaults to 0.801. Because this adaptation is stochastic the realized average during the main sampling phase can often vary between 0.75 and 0.85.

So long as the target distribution is sufficiently well-behaved then the adaptation should always converge to that target, at least for long enough warmup periods. Small averages indicate some obstruction to the adaptation, for example discontinuities in the target distribution or inaccurate gradient evaluations.

```
# Check all Hamiltonian Monte Carlo Diagnostics
# for an ensemble of Markov chains
# @param diagnostics A named list of two-dimensional arrays for
#                    each diagnostic. The first dimension of each
#                    element indexes the Markov chains and the
#                    second dimension indexes the sequential
#                    states within each Markov chain.
# @param adapt_target Target acceptance proxy statistic for step size
#                    adaptation.
# @param max_treedepth The maximum numerical trajectory treedepth
# @param max_width Maximum line width for printing
check_all_hmc_diagnostics <- function(diagnostics,
                                     adapt_target=0.801,
                                     max_treedepth=10,
                                     max_width=72) {
  validate_named_list_of_arrays(diagnostics, 'diagnostics')

  no_warning <- TRUE
  no_divergence_warning <- TRUE
  no_treedepth_warning <- TRUE
  no_efmi_warning <- TRUE
  no_accept_warning <- TRUE

  message <- ""

  C <- dim(diagnostics[['divergent__']])[1]
  S <- dim(diagnostics[['divergent__']])[2]

  for (c in 1:C) {
    local_message <- ""
```

```

# Check for divergences
n_div <- sum(diagnostics[['divergent__']][c,])

if (n_div > 0) {
  no_warning <- FALSE
  no_divergence_warning <- FALSE
  local_message <-
    paste0(local_message,
      sprintf(' Chain %s: %s of %s transitions (0.1f%%) ',
        c, n_div, S, 100 * n_div / S),
      'diverged.\n')
}

# Check for tree depth saturation
n_tds <- sum(sapply(diagnostics[['treedepth__']][c,],
  function(s) s >= max_treedepth))

if (n_tds > 0) {
  no_warning <- FALSE
  no_treedepth_warning <- FALSE
  local_message <-
    paste0(local_message,
      sprintf(' Chain %s: %s of %s transitions (0.2f%%)\n',
        c, n_tds, S, 100 * n_tds / S),
      '
      saturated the maximum treedepth ',
      sprintf('of %s.\n', max_treedepth))
}

# Check the energy fraction of missing information (E-FMI)
energies = diagnostics[['energy__']][c,]
numer = sum(diff(energies)**2) / length(energies)
denom = var(energies)
efmi <- numer / denom
if (efmi < 0.2) {
  no_warning <- FALSE
  no_efmi_warning <- FALSE
  local_message <-
    paste0(local_message,
      sprintf(' Chain %s: E-FMI = 0.3f.\n', c, efmi))
}

# Check convergence of the stepsize adaptation

```

```

ave_accept_proxy <- mean(diagnostics[['accept_stat__']][c,])
if (ave_accept_proxy < 0.9 * adapt_target) {
  no_warning <- FALSE
  no_accept_warning <- FALSE
  local_message <-
    paste0(local_message,
            sprintf(' Chain %s: Average proxy acceptance ', c),
            sprintf('statistic (%.3f)\n', ave_accept_proxy),
            '
            is smaller than 90% of the target ',
            sprintf(' (%.3f).\n', adapt_target))
}

if (local_message != "") {
  message <- paste0(message, local_message, '\n')
}
}

if (no_warning) {
  desc <- paste0('All Hamiltonian Monte Carlo diagnostics are ',
                 'consistent with reliable Markov chain Monte Carlo.')
  desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
  message <- paste0(message, desc)
}

if (!no_divergence_warning) {
  desc <- paste0('Divergent Hamiltonian transitions result from ',
                 'unstable numerical trajectories. These ',
                 'instabilities are often due to degenerate target ',
                 'geometry, especially "pinches". If there are ',
                 'only a small number of divergences then running ',
                 'with adept_delta larger ',
                 sprintf('than %.3f may reduce the ', adapt_target),
                 'instabilities at the cost of more expensive ',
                 'Hamiltonian transitions.')
  desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
  message <- paste0(message, desc, '\n\n')
}

if (!no_treedepth_warning) {
  desc <- paste0('Numerical trajectories that saturate the ',
                 'maximum treedepth have terminated prematurely. ',
                 sprintf('Increasing max_depth above %s ', max_treedepth),

```

```

        'should result in more expensive, but more ',
        'efficient, Hamiltonian transitions.')
    desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
    message <- paste0(message, desc, '\n\n')
}

if (!no_efmi_warning) {
  desc <- paste0('E-FMI below 0.2 arise when a funnel-like geometry ',
    'obstructs how effectively Hamiltonian trajectories ',
    'can explore the target distribution.')
  desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
  message <- paste0(message, desc, '\n\n')
}

if (!no_accept_warning) {
  desc <- paste0('A small average proxy acceptance statistic ',
    'indicates that the adaptation of the numerical ',
    'integrator step size failed to converge. This is ',
    'often due to discontinuous or imprecise ',
    'gradients.')
  desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
  message <- paste0(message, desc, '\n\n')
}

cat(message)
}

```

2.2 Integrator Inverse Metric Elements

Diagnostic failures indicate the presence of problems but only hint at the nature of those problems. In order to resolve the underlying problems we need to investigate them beyond these hints. Fortunately Hamiltonian Monte Carlo provides a wealth of additional information that can assist.

First we can look at the inverse metric adaptation in each of the Markov chains. Inconsistencies in the adapted inverse metric elements across the Markov chains are due to the individual chains encountering different behaviors during warmup.

```

# Plot outcome of inverse metric adaptation
# @params stan_fit A StanFit object
# @params B The number of bins for the inverse metric element histograms.
plot_inv_metric <- function(stan_fit, B=25) {

```



```

adaptation_info <- rstan:::get_adaptation_info(stan_fit)
C <- length(adaptation_info)

inv_metric_elems <- list()
for (c in 1:C) {
  raw_info <- adaptation_info[[c]]
  clean1 <- sub("# Adaptation terminated\n# Step size = [0-9.]*\n#",
    "", raw_info)
  clean2 <- sub(" [a-zA-Z]*:\n# ", "", clean1)
  clean3 <- sub("\n$", "", clean2)
  inv_metric_elems[[c]] <- as.numeric(strsplit(clean3, ',')[[1]])
}

min_elem <- min(unlist(inv_metric_elems))
max_elem <- max(unlist(inv_metric_elems))

delta <- (max_elem - min_elem) / B
min_elem <- min_elem - delta
max_elem <- max_elem + delta
bins <- seq(min_elem, max_elem, delta)
B <- B + 2

max_y <- max(sapply(1:C, function(c)
  max(hist(inv_metric_elems[[c]], breaks=bins, plot=FALSE)$counts)))

idx <- rep(1:B, each=2)
x <- sapply(1:length(idx), function(b) if(b %% 2 == 1) bins[idx[b]]
  else bins[idx[b] + 1])

par(mfrow=c(2, 2), mar = c(5, 2, 2, 1))
colors <- c(c_dark, c_mid_highlight, c_mid, c_light_highlight)

for (c in 1:C) {
  counts <- hist(inv_metric_elems[[c]], breaks=bins, plot=FALSE)$counts
  y <- counts[idx]

  plot(x, y, type="l", main=paste("Chain", c), col=colors[c],
    xlim=c(min_elem, max_elem), xlab="Inverse Metric Elements",
    ylim=c(0, 1.05 * max_y), ylab="", yaxt="n")
}
}

```

Note that the adaptation information may be accessed differently in other Stan interfaces, in which case this function would have to be modified accordingly.

2.3 Integrator Step Sizes

The other product of Stan's adaptation is the step size of the numerical integrator used to build the numerical Hamiltonian trajectories. As with the inverse metric elements heterogeneity in the adapted values across the Markov chains indicates that the Markov chains encountered substantially different behavior during warmup.

```
# Display adapted symplectic integrator step sizes
# @param diagnostics A named list of two-dimensional arrays for
#                   each diagnostic. The first dimension of each
#                   element indexes the Markov chains and the
#                   second dimension indexes the sequential
#                   states within each Markov chain.
display_stepsizes <- function(diagnostics) {
  validate_named_list_of_arrays(diagnostics, 'diagnostics')

  stepsizes <- diagnostics[['stepsize_']]
  C <- dim(stepsizes)[1]

  for (c in 1:C) {
    stepsize <- stepsizes[c, 1]
    cat(sprintf('Chain %s: Integrator Step Size = %f\n',
                c, stepsize))
  }
}
```

2.4 Numerical Trajectory Lengths

We can see the consequence of the adapted step sizes by looking at the numerical trajectories generated for each Hamiltonian Markov transition. The longer these trajectories the more degenerate the target distribution, and the more expensive it is to explore.

```
# Display symplectic integrator trajectory lengths
# @param diagnostics A named list of two-dimensional arrays for
#                   each diagnostic. The first dimension of each
#                   element indexes the Markov chains and the
#                   second dimension indexes the sequential
#                   states within each Markov chain.
```

```

plot_num_leapfrogs <- function(diagnostics) {
  validate_named_list_of_arrays(diagnostics, 'diagnostics')

  lengths <- diagnostics[['n_leapfrog_']]
  C <- dim(lengths)[1]

  max_length <- max(lengths) + 1
  max_count <- max(sapply(1:C, function(c) max(table(lengths[c,]))))

  colors <- c(c_dark, c_mid_highlight, c_mid, c_light_highlight)

  idx <- rep(1:max_length, each=2)
  xs <- sapply(1:length(idx), function(b) if(b %% 2 == 0) idx[b] + 0.5
                                                    else idx[b] - 0.5)

  plot(0, type="n",
       xlab="Numerical Trajectory Length",
       xlim=c(0.5, max_length + 0.5),
       ylab="", ylim=c(0, 1.1 * max_count), yaxt='n')

  for (c in 1:C) {
    counts <- hist(lengths[c,],
                   seq(0.5, max_length + 0.5, 1),
                   plot=FALSE)$counts
    pad_counts <- counts[idx]
    lines(xs, pad_counts, lwd=2, col=colors[c])
  }
}

```

```

# Display symplectic integrator trajectory lengths by Markov chain
# @param diagnostics A named list of two-dimensional arrays for
#                    each diagnostic. The first dimension of each
#                    element indexes the Markov chains and the
#                    second dimension indexes the sequential
#                    states within each Markov chain.
plot_num_leapfrogs_by_chain <- function(diagnostics) {
  validate_named_list_of_arrays(diagnostics, 'diagnostics')

  lengths <- diagnostics[['n_leapfrog_']]
  C <- dim(lengths)[1]

  max_length <- max(lengths) + 1

```

```

max_count <- max(sapply(1:C, function(c) max(table(lengths[c,]))))

colors <- c(c_dark, c_mid_highlight, c_mid, c_light_highlight)

idx <- rep(1:max_length, each=2)
xs <- sapply(1:length(idx), function(b) if(b %% 2 == 0) idx[b] + 0.5
                                         else idx[b] - 0.5)

par(mfrow=c(2, 2), mar = c(5, 2, 2, 1))

for (c in 1:C) {
  stepsize <- round(diagnostics[['stepsize_']][c,1], 3)

  counts <- hist(lengths[c,],
                  seq(0.5, max_length + 0.5, 1),
                  plot=FALSE)$counts
  pad_counts <- counts[idx]

  plot(xs, pad_counts, type="l", lwd=2, col=colors[c],
        main=paste0("Chain ", c, " (Stepsize = ", stepsize, ")"),
        xlab="Numerical Trajectory Length", xlim=c(0.5, max_length + 0.5),
        ylab="", ylim=c(0, 1.1 * max_count), yaxt='n')
}
}

```

2.5 Average Proxy Acceptance Statistic

When the different adaptation outcomes are due to problematic behaviors encountered during warmup then the average proxy acceptance statistics should also vary across the Markov chains.

```

# Display empirical average of the proxy acceptance statistic across
# each Markov chain
# @param diagnostics A named list of two-dimensional arrays for
#                     each diagnostic. The first dimension of each
#                     element indexes the Markov chains and the
#                     second dimension indexes the sequential
#                     states within each Markov chain.
display_ave_accept_proxy <- function(diagnostics) {
  validate_named_list_of_arrays(diagnostics, 'diagnostics')

```

```

proxy_stats <- diagnostics[['accept_stat__']]
C <- dim(proxy_stats)[1]

for (c in 1:C) {
  ave_accept_proxy <- mean(proxy_stats[c,])
  cat(sprintf('Chain %s: Average proxy acceptance statistic = %.3f\n',
              c, ave_accept_proxy))
}
}

```

2.6 Divergence-Labeled Pairs Plot

One of the most powerful features of divergent transitions is that they not only indicate problematic geometry but also provide some spatial information on the source of that problematic geometry. In particular the states generated from unstable numerical Hamiltonian trajectories will tend to be closer to the problematic geometry than those from stable trajectories.

Consequently if we plot the states from divergent and non-divergent transitions separately then we should see the divergent states concentrate towards the problematic behavior. The high-dimensional states themselves can be visualized with pairs plots.

```

# Apply transformation identity, log, or logit transformation to
# named values and flatten the output. Transformation defaults to
# identity if name is not included in `transforms` dictionary. A
# ValueError is thrown if values are not properly constrained.
# @param name Expectand name.
# @param expectand_vals_list A named list of two-dimensional arrays for
#                             each expectand. The first dimension of
#                             each element indexes the Markov chains and
#                             the second dimension indexes the sequential
#                             states within each Markov chain.
# @param transforms A named list of transformation flags for each
#                   expectand.
# @return The transformed expectand name and a one-dimensional array of
#         flattened transformation outputs.
apply_transform <- function(name, expectand_vals_list, transforms) {
  t <- transforms[[name]]
  if (is.null(t)) t <- 0

  transformed_name <- ""
  transformed_vals <- 0

```

```

if (t == 0) {
  transformed_name <- name
  transformed_vals <- c(t(expectand_vals_list[[name]]),
                        recursive=TRUE)
} else if (t == 1) {
  if (min(expectand_vals_list[[name]]) <= 0) {
    stop(paste0('Log transform requested for expectand ',
                sprintf('%s ', name),
                'but expectand values are not strictly positive.'))
  }
  transformed_name <- paste0('log(', name, ')')
  transformed_vals <- log(c(t(expectand_vals_list[[name]]),
                            recursive=TRUE))
} else if (t == 2) {
  if (min(expectand_vals_list[[name]]) <= 0 |
      max(expectand_vals_list[[name]] >= 1)) {
    stop(paste0('Logit transform requested for expectand ',
                sprintf('%s ', name),
                'but expectand values are not strictly confined ',
                'to the unit interval.'))
  }
  transformed_name <- paste0('logit(', name, ')')
  transformed_vals <- sapply(c(t(expectand_vals_list[[name]]),
                                recursive=TRUE),
                             function(x) log(x / (1 - x)))
}
return (list('t_name' = transformed_name,
             't_vals' = transformed_vals))
}

```

```

# Plot pairwise scatter plots with non-divergent and divergent
# transitions separated by color
# @param x_names An array of expectand names to be plotted on the x axis.
# @param y_names An array of expectand names to be plotted on the y axis.
# @param expectand_vals_list A named list of two-dimensional arrays for
#                             each expectand. The first dimension of
#                             each element indexes the Markov chains and
#                             the second dimension indexes the sequential
#                             states within each Markov chain.
# @param diagnostics A named list of two-dimensional arrays for
#                     each diagnostic. The first dimension of each
#                     element indexes the Markov chains and the

```

```

# second dimension indexes the sequential
# states within each Markov chain.
# @params transforms A named list of flags configuring which if any
# transformation to apply to each named expectand:
# 0: identity
# 1: log
# 2: logit
# @param xlim Optional global x-axis bounds for all pair plots.
# Defaults to dynamic bounds for each pair plot.
# @param ylim Optional global y-axis bounds for all pair plots.
# Defaults to dynamic bounds for each pair plot.
# @params plot_mode Plotting style configuration:
# 0: Non-divergent transitions are plotted in
# transparent red while divergent transitions are
# plotted in transparent green.
# 1: Non-divergent transitions are plotted in gray
# while divergent transitions are plotted in
# different shades of teal depending on the
# trajectory length. Transitions from shorter
# trajectories should cluster somewhat closer to
# the neighborhoods with problematic geometries.
# @param max_width Maximum line width for printing
plot_div_pairs <- function(x_names, y_names,
                           expectand_vals_list, diagnostics,
                           transforms=list(), xlim=NULL, ylim=NULL,
                           plot_mode=0, max_width=72) {
  if (!is.vector(x_names)) {
    stop('Input variable `x_names` is not an array.')
  }

  if (!is.vector(y_names)) {
    stop('Input variable `y_names` is not an array.')
  }

  validate_named_list_of_arrays(expectand_vals_list,
                                'expectand_vals_list')

  validate_named_list_of_arrays(diagnostics, 'diagnostics')

  if (length(transforms) > 0)
    validate_named_list(transforms, 'transforms')

```

```

# Check transform flags
for (name in names(transforms)) {
  if (transforms[[name]] < 0 | transforms[[name]] > 2) {
    warning <-
      paste0(sprintf('The transform flag %s for expectand %s ',
                     transforms[[name]], name),
              'is invalid. Plot will default to no transformation.')
    warning <- paste0(strwrap(warning, max_width, 0), collapse='\n')
    cat(warning)
  }
}

# Check plot mode
if (plot_mode < 0 | plot_mode > 1) {
  stop(sprintf('Invalid `plot_mode` value %s.', plot_mode))
}

# Transform expectand values
transformed_vals = list()

transformed_x_names <- c()
for (name in x_names) {
  r <- apply_transform(name, expectand_vals_list, transforms)
  if (is.null(r))
    stop()
  transformed_x_names <- c(transformed_x_names, r$t_name)
  if (! r$t_name %in% transformed_vals) {
    transformed_vals[[r$t_name]] <- r$t_vals
  }
}

transformed_y_names <- c()
for (name in y_names) {
  r <- apply_transform(name, expectand_vals_list, transforms)
  if (is.null(r))
    stop()
  transformed_y_names <- c(transformed_y_names, r$t_name)
  if (! r$t_name %in% transformed_vals) {
    transformed_vals[[r$t_name]] <- r$t_vals
  }
}

```



```

# Create pairs of transformed expectands, dropping duplicates
pairs <- list()
for (x_name in transformed_x_names) {
  for (y_name in transformed_y_names) {
    if (x_name == y_name) next
    if (any(sapply(pairs, identical, c(x_name, y_name)))) next
    if (any(sapply(pairs, identical, c(y_name, x_name)))) next
    pairs[[length(pairs) + 1]] <- c(x_name, y_name)
  }
}

# Extract non-divergent and divergent transition indices
divs <- diagnostics[['divergent__']]
C <- dim(divs)[1]
nondiv_filter <- c(sapply(1:C, function(c) divs[c,] == 0))
div_filter <- c(sapply(1:C, function(c) divs[c,] == 1))

if (plot_mode == 1) {
  if (sum(div_filter) > 0) {
    nlfs <- c(sapply(1:C,
                     function(c) diagnostics[['n_leapfrog__']][c,]))
    div_nlfs <- nlfs[div_filter]
    max_nlf <- max(div_nlfs)
    nom_colors <- c(c_light_teal, c_mid_teal, c_dark_teal)
    cmap <- colormap(colormap=nom_colors, nshades=max_nlf)
  } else {
    div_nlfs <- c()
    nom_colors <- c(c_light_teal, c_mid_teal, c_dark_teal)
    cmap <- colormap(colormap=nom_colors, nshades=1)
  }
}

# Set plot layout dynamically
N_cols <- 3
N_plots <- length(pairs)
if (N_plots <= 3) {
  par(mfrow=c(1, N_plots), mar = c(5, 5, 2, 1))
} else if (N_plots == 4) {
  par(mfrow=c(2, 2), mar = c(5, 5, 2, 1))
} else if (N_plots == 6) {
  par(mfrow=c(2, 3), mar = c(5, 5, 2, 1))
} else {

```

```

    par(mfrow=c(3, N_cols), mar = c(5, 5, 2, 1))
  }

  # Plot
  c_dark_trans <- c("#8F272780")
  c_green_trans <- c("#00FF0080")

  for (pair in pairs) {
    x_name <- pair[1]
    x_nondiv_vals <- transformed_vals[[x_name]][nondiv_filter]
    x_div_vals <- transformed_vals[[x_name]][div_filter]

    if (is.null(xlim)) {
      xmin = min(transformed_vals[[x_name]])
      xmax = max(transformed_vals[[x_name]])
      local_xlim <- c(xmin, xmax)
    } else {
      local_xlim <- xlim
    }

    y_name <- pair[2]
    y_nondiv_vals <- transformed_vals[[y_name]][nondiv_filter]
    y_div_vals <- transformed_vals[[y_name]][div_filter]

    if (is.null(ylim)) {
      ymin = min(transformed_vals[[y_name]])
      ymax = max(transformed_vals[[y_name]])
      local_ylim <- c(ymin, ymax)
    } else {
      local_ylim <- ylim
    }

    if (plot_mode == 0) {
      plot(x_nondiv_vals, y_nondiv_vals,
           col=c_dark_trans, pch=16, main="",
           xlab=x_name, xlim=local_xlim,
           ylab=y_name, ylim=local_ylim)
      points(x_div_vals, y_div_vals,
             col=c_green_trans, pch=16)
    }
    if (plot_mode == 1) {
      plot(x_nondiv_vals, y_nondiv_vals,

```

```

        col="#DDDDDD", pch=16, main="",
        xlab=x_name, xlim=local_xlim,
        ylab=y_name, ylim=local_ylim)
    points(x_div_vals, y_div_vals,
           col=cmap[div_nlf], pch=16)
  }
}
}

```

3 Expectand Diagnostic Functions

The Hamiltonian Monte Carlo diagnostics exploited the particular structure of the Hamiltonian Markov transition. For a general Markov transition we don't have any particular structure to exploit, and hence limited diagnostic options. In this general setting we have to investigate the behavior of not the entire state but instead particular expectands of interest.

3.1 xihat

A Markov chain Monte Carlo central limit theorem cannot exist for the expectand $f : X \rightarrow \mathbb{R}$ unless both $\mathbb{E}_\pi[f]$ and $\mathbb{E}_\pi[f^2]$ are finite, in which case we say that the expectand is sufficiently integrable. Moreover the smaller the following moments the faster the central limit theorem will kick in.

$\hat{\xi}$ uses the tail behavior of a realized Markov chain to estimate the integrability of an expectand. More specifically $\hat{\xi}$ estimates the shape of a general Pareto density function from non-central values of the expectand.

If the tail behavior were exactly general Pareto then the larger the shape parameter ξ the fewer moments of the distribution will be well-defined. Formally the m th-order moment is well-defined only if

$$m < \frac{1}{\xi}.$$

For example with $\xi = 0.9$ the expectation $\mathbb{E}_\pi[f]$ is finite but $\mathbb{E}_\pi[f^2]$ is not. Similarly for $\xi = 0.4$ the expectations $\mathbb{E}_\pi[f]$ and $\mathbb{E}_\pi[f^2]$ are finite but the third-order moment $\mathbb{E}_\pi[f^3]$ is not.

The estimator $\hat{\xi}$ is constructed from the smallest and largest values of an expectand evaluated across a realized Markov chain, where the smallest and largest values are separated from the central values using a heuristic. Because $\hat{\xi}$ only estimates the tail shape I require a conservative threshold of $\hat{\xi} \geq 0.25$ for the diagnostic warning to be triggered.

If the expectand output is bounded then the lower and upper tail might consist of the same value. In this case the $\hat{\xi}$ estimator is poorly-behaved, but the boundedness also guarantees

that moments of all orders exist. To make this diagnostic as robust as possible $\hat{\xi}$ will return -2 in these cases to avoid the diagnostic threshold.

```
# Compute hat{xi}, an estimate for the shape of a generalized Pareto
# distribution from a sample of positive values using the method
# introduced in "A New and Efficient Estimation Method for the
# Generalized Pareto Distribution" by Zhang and Stephens
# https://doi.org/10.1198/tech.2009.08017.
#
# Within the generalized Pareto distribution family all moments up to
# the mth order are finite if and only if
#   xi < 1 / m.
#
# @params vals A one-dimensional array of positive values.
# @return Shape parameter estimate.
compute_xi_hat <- function(vals) {
  N <- length(vals)
  sorted_vals <- sort(vals)

  # Return erroneous result if all input values are the same
  if (sorted_vals[1] == sorted_vals[N]) {
    return (NaN)
  }

  # Return erroneous result if all input values are not positive
  if (sorted_vals[1] < 0) {
    cat("Input values must be positive.")
    return (NaN)
  }

  # Estimate 25% quantile
  q <- sorted_vals[floor(0.25 * N + 0.5)]

  if (q == sorted_vals[1]) {
    return (-2)
  }

  # Heuristic generalized Pareto shape configuration
  M <- 20 + floor(sqrt(N))

  b_hat_vec <- rep(0, M)
  log_w_vec <- rep(0, M)
```

```

for (m in 1:M) {
  b_hat_vec[m] <- 1 / sorted_vals[N] +
    (1 - sqrt(M / (m - 0.5))) / (3 * q)
  if (b_hat_vec[m] != 0) {
    xi_hat <- mean( log(1 - b_hat_vec[m] * sorted_vals) )
    log_w_vec[m] <- N * ( log(-b_hat_vec[m] / xi_hat) - xi_hat - 1 )
  } else {
    log_w_vec[m] <- 0
  }
}

# Remove terms that don't contribute to average to improve numerical
# stability
log_w_vec <- log_w_vec[b_hat_vec != 0]
b_hat_vec <- b_hat_vec[b_hat_vec != 0]

max_log_w <- max(log_w_vec)
b_hat <- sum(b_hat_vec * exp(log_w_vec - max_log_w)) /
  sum(exp(log_w_vec - max_log_w))

mean( log (1 - b_hat * sorted_vals) )
}

```

```

# Compute empirical generalized Pareto shape for upper and lower tails
# for the given expectand values, ignoring any autocorrelation between
# the values.
# @param vals A one-dimensional array of sequential expectand values.
# @return Left and right shape estimators.
compute_tail_xi_hats <- function(vals) {
  v_center <- median(vals)

  # Isolate lower and upper tails which can be adequately modeled by a
  # generalized Pareto shape for sufficiently well-behaved distributions
  vals_left <- abs(vals[vals < v_center] - v_center)
  N <- length(vals_left)
  M <- min(0.2 * N, 3 * sqrt(N))
  vals_left <- vals_left[M:N]

  vals_right <- vals[vals > v_center] - v_center
  N <- length(vals_right)
  M <- min(0.2 * N, 3 * sqrt(N))
  vals_right <- vals_right[M:N]
}

```

```

# Default to NaN if left tail is ill-defined
xi_hat_left <- NaN
if (length(vals_left) > 40)
  xi_hat_left <- compute_xi_hat(vals_left)

# Default to NaN if right tail is ill-defined
xi_hat_right <- NaN
if (length(vals_right) > 40)
  xi_hat_right <- compute_xi_hat(vals_right)

c(xi_hat_left, xi_hat_right)
}

# Check upper and lower tail behavior of the expectand values.
# @param expectand_vals A two-dimensional array of expectand values with
#                       the first dimension indexing the Markov chains
#                       and the second dimension indexing the sequential
#                       states within each Markov chain.
# @param max_width Maximum line width for printing
check_tail_xi_hats <- function(expectand_vals, max_width=72) {
  validate_array(expectand_vals, 'expectand_vals')
  C <- dim(expectand_vals)[1]

  no_warning <- TRUE
  message <- ""

  for (c in 1:C) {
    xi_hats <- compute_tail_xi_hats(expectand_vals[c,])
    xi_hat_threshold <- 0.25
    if ( is.nan(xi_hats[1]) & is.nan(xi_hats[2]) ) {
      no_warning <- FALSE
      body <- ' Chain %s: Both left and right hat{xi}s are NaN.\n'
      message <- paste0(message, sprintf(body, c))
    }
    else if ( is.nan(xi_hats[1]) ) {
      no_warning <- FALSE
      body <- ' Chain %s: Left hat{xi} is NaN.\n'
      message <- paste0(message, sprintf(body, c))
    } else if ( is.nan(xi_hats[2]) ) {
      no_warning <- FALSE
      body <- ' Chain %s: Right hat{xi} is NaN.\n'
      message <- paste0(message, sprintf(body, c))
    }
  }
}

```

```

} else if (xi_hats[1] >= xi_hat_threshold &
  xi_hats[2] >= xi_hat_threshold) {
  no_warning <- FALSE
  body <- paste0(' Chain %s: Both left and right tail ',
    'hat{xi}s (%.3f, %.3f) exceed %.2f.\n')
  message <- paste0(message, sprintf(body, c,
    xi_hats[1], xi_hats[2],
    xi_hat_threshold))
} else if (xi_hats[1] < xi_hat_threshold &
  xi_hats[2] >= xi_hat_threshold ) {
  no_warning <- FALSE
  body <- paste0(' Chain %s: Right tail hat{xi} (%.3f) ',
    'exceeds %.2f.\n')
  message <- paste0(message, sprintf(body, c, xi_hats[2],
    xi_hat_threshold))
} else if (xi_hats[1] >= xi_hat_threshold &
  xi_hats[2] < xi_hat_threshold ) {
  no_warning <- FALSE
  body <- paste0(' Chain %s: Left tail hat{xi} (%.3f) ',
    'exceeds %.2f.\n')
  message <- paste0(message, sprintf(body, c, xi_hats[1],
    xi_hat_threshold))
}
}

if (no_warning) {
  desc <- 'Expectand appears to be sufficiently integrable.\n\n'
  message <- paste0(message, desc)
} else {
  desc <- paste0('Large tail xi_hats suggest that the expectand ',
    'might not be sufficiently integrable.\n\n')
  desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
  message <- paste0(message, desc)
}

cat(message)
}

```

3.2 Frozen Chains

Another sign of problems is when all evaluations of an expectand are constant. This could be due to the Markov chain being stuck at a single state or just that the pushforward distribution of the expectand concentrates on a single value. We can't distinguish between these possibilities without more information, but we can flag a constant expectand by looking at its empirical variance.

Here we'll use a Welford accumulator to compute the empirical variance of the expectand values in a single sweep.

```
# Compute empirical mean and variance of a given sequence with a single
# pass using Welford accumulators.
# @params vals A one-dimensional array of sequential expectand values.
# @return The empirical mean and variance.
welford_summary <- function(vals) {
  mean <- 0
  var <- 0

  N <- length(vals)
  for (n in 1:N) {
    delta <- vals[n] - mean
    mean <- mean + delta / n
    var <- var + delta * (vals[n] - mean)
  }

  var <- var / (N - 1)

  return(c(mean, var))
}
```

```
# Check expectand values for vanishing empirical variance.
# @param expectand_vals A two-dimensional array of expectand values with
#                       the first dimension indexing the Markov chains
#                       and the second dimension indexing the sequential
#                       states within each Markov chain.
# @param max_width Maximum line width for printing
check_variances <- function(expectand_vals, max_width=72) {
  validate_array(expectand_vals, 'expectand_vals')
  C <- dim(expectand_vals)[1]

  no_warning <- TRUE
  message <- ""
```



```

for (c in 1:C) {
  var <- welford_summary(expectand_vals[c,])[2]
  if (var < 1e-10) {
    body <- ' Chain %s: Expectand values are constant.\n'
    message <- paste0(message, sprintf(body, c))
    no_warning <- FALSE
  }
}

if (no_warning) {
  desc <- 'Expectand values are varying across all Markov chains.\n\n'
  message <- paste0(message, desc)
} else {
  desc <- paste0('If the expectand is not expected to be nearly ',
                 'constant then the Markov transition might be ',
                 'misbehaving.\n\n')
  desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
  message <- paste0(message, desc)
}

cat(message)
}

```

3.3 Split Rhat

One of the key features of Markov chain equilibrium is that the distribution of Markov chain realizations is independent of the initialization. In particular the expectand evaluations from any equilibrated Markov chain should be statistically equivalent to any other. Even more the evaluations across any subset of Markov chain states should be equivalent.

The split $\hat{R}$ statistic quantifies the heterogeneity in the expectand evaluations across an ensemble of Markov chains, each of which has been split in half. Mathematically split $\hat{R}$ is similar to analysis of variance in that compares the empirical variance of the average expectand values in each chain half to the average of the empirical variances in each chain half; the key difference is that split $\hat{R}$ transforms this ratio so that in equilibrium the statistic decays towards 1 from above.

When split $\hat{R}$ is much larger than 1 the expectand evaluations across each Markov chain halves are not consistent with each other. This could be because the Markov chains have not converged to the same typical set or because they have not yet expanded into that typical set.

```

# Split a sequence of expectand values in half to create an initial and
# terminal Markov chains
# @params chain A sequence of expectand values derived from a single
#               Markov chain.
# @return Two subsequences of expectand values.
split_chain <- function(chain) {
  N <- length(chain)
  M <- N %/% 2
  list(chain1 <- chain[1:M], chain2 <- chain[(M + 1):N])
}

```

```

# Compute split hat{R} for the expectand values.
# @param expectand_vals A two-dimensional array of expectand values with
#                       the first dimension indexing the Markov chains
#                       and the second dimension indexing the sequential
#                       states within each Markov chain.
# @return Split Rhat estimate.
compute_split_rhat <- function(expectand_vals) {
  validate_array(expectand_vals, 'expectand_vals')
  C <- dim(expectand_vals)[1]

  split_chain_vals <- unlist(lapply(1:C,
                                   function(c)
                                     split_chain(expectand_vals[c,])),
                             recursive=FALSE)

  N_chains <- length(split_chain_vals)
  N <- sum(sapply(1:C, function(c) length(expectand_vals[c,])))

  means <- rep(0, N_chains)
  vars <- rep(0, N_chains)

  for (c in 1:N_chains) {
    summary <- welford_summary(split_chain_vals[[c]])
    means[c] <- summary[1]
    vars[c] <- summary[2]
  }

  total_mean <- sum(means) / N_chains
  W = sum(vars) / N_chains
  B = N * sum(sapply(means, function(m)
                      (m - total_mean)**2)) / (N_chains - 1)

```

```

rhat = NaN
if (abs(W) > 1e-10)
  rhat = sqrt( (N - 1 + B / W) / N )

(rhat)
}

```

```

# Compute split hat{R} for all input expectands.
# @param expectand_vals_list A named list of two-dimensional arrays for
#                             each expectand. The first dimension of
#                             each element indexes the Markov chains and
#                             the second dimension indexes the sequential
#                             states within each Markov chain.
# @return Array of split Rhat estimates.
compute_split_rhats <- function(expectand_vals_list) {
  validate_named_list_of_arrays(expectand_vals_list,
                                'expectand_vals_list')

  rhats <- c()
  for (name in names(expectand_vals_list)) {
    expectand_vals <- expectand_vals_list[[name]]
    rhats <- c(rhats, compute_split_rhat(expectand_vals))
  }
  return(rhats)
}

```

```

# Check split hat{R} across the given expectand values.
# @param expectand_vals A two-dimensional array of expectand values with
#                       the first dimension indexing the Markov chains
#                       and the second dimension indexing the sequential
#                       states within each Markov chain.
# @param max_width Maximum line width for printing
check_rhat <- function(expectand_vals, max_width=72) {
  validate_array(expectand_vals, 'expectand_vals')

  rhat <- compute_split_rhat(expectand_vals)

  no_warning <- TRUE
  message <- ""

  if (is.nan(rhat)) {

```

```

    message <- paste0(message,
                      'All Markov chains appear to be frozen.\n')
  } else if (rhat > 1.1) {
    message <- paste0(message, sprintf('Split hat{R} is %f.\n', rhat))
    no_warning <- FALSE
  }

  if (no_warning) {
    desc <- 'Markov chain behavior is consistent with equilibrium.\n\n'
    message <- paste0(message, desc)
  } else {
    desc <- paste0('Split hat{R} larger than 1.1 suggests that at ',
                  'least one of the Markov chains has not reached ',
                  'an equilibrium.\n\n')
    desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
    message <- paste0(message, desc)
  }

  cat(message)
}

```

3.4 Integrated Autocorrelation Time

The information about the target distribution encoded within a Markov chain, and hence the potential precision of Markov chain Monte Carlo estimators, is limited by the autocorrelation of the internal states. Assuming equilibrium we can estimate the stationary autocorrelations between the outputs of a given expectand from the realized Markov chain and then combine them into an estimate of the integrated autocorrelation time $\hat{\tau}[f]$.

```

# Compute empirical integrated autocorrelation time, \hat{\tau}, for a
# sequence of expectand values.
# @param vals A one-dimensional array of sequential expectand values.
# @return Left and right shape estimators.
compute_tau_hat <- function(vals) {
  # Compute empirical autocorrelations
  N <- length(vals)
  zs <- vals - mean(vals)

  B <- 2**ceiling(log2(N)) # Next power of 2 after N
  zs_buff <- c(zs, rep(0, B - N))

```

```

Fs <- fft(zs_buff)
Ss <- Fs * Conj(Fs)
Rs <- fft(Ss, inverse=TRUE)

acov_buff <- Re(Rs)

if (acov_buff[1] == 0)
  return(Inf)

rhos <- head(acov_buff, N) / acov_buff[1]

# Drop last lag if (L + 1) is odd so that the lag pairs are complete
L <- N
if ((L + 1) %% 2 == 1)
  L <- L - 1

# Number of lag pairs
P <- (L + 1) / 2

# Construct asymptotic correlation from initial monotone sequence
old_pair_sum <- rhos[1] + rhos[2]
for (p in 2:P) {
  current_pair_sum <- rhos[2 * p - 1] + rhos[2 * p]

  if (current_pair_sum < 0) {
    rho_sum <- sum(rhos[2:(2 * p)])

    if (rho_sum <= -0.25)
      rho_sum <- -0.25

    asymp_corr <- 1.0 + 2 * rho_sum
    return (asymp_corr)
  }

  if (current_pair_sum > old_pair_sum) {
    current_pair_sum <- old_pair_sum
    rhos[2 * p - 1] <- 0.5 * old_pair_sum
    rhos[2 * p] <- 0.5 * old_pair_sum
  }

  if (p == P) {
    return (NaN)
  }
}

```

```

    }

    old_pair_sum <- current_pair_sum
  }
}

```

This, estimate, however, can be unreliable if the Markov chains have not had sufficient time to explore. In my experience a good rule of thumb is that the empirical integrated autocorrelation time has cannot be larger than five times the number of total iterations,

$$\hat{\tau}[f] < 5 \cdot N.$$

Equivalently the incremental empirical integrated autocorrelation time cannot be larger than five,

$$\frac{\hat{\tau}[f]}{N} < 5.$$

```

# Check the incremental empirical integrated autocorrelation time for
# all the given expectand values.
# @param expectand_vals A two-dimensional array of expectand values with
#                        the first dimension indexing the Markov chains
#                        and the second dimension indexing the sequential
#                        states within each Markov chain.
# @param max_width Maximum line width for printing
check_inc_tau_hat <- function(expectand_vals,
                              max_width=72) {
  validate_array(expectand_vals, 'expectand_vals')

  C <- dim(expectand_vals)[1]
  S <- dim(expectand_vals)[2]

  no_warning <- TRUE
  message <- ""

  for (c in 1:C) {
    tau_hat <- compute_tau_hat(expectand_vals[c,])

    if (is.infinite(tau_hat)) {
      no_warning <- FALSE
      body <- paste0(' Chain %s: The calculation of hat{tau} is ',
                     'unreliable because\nthe empirical variance ',
                     'is zero.\n')
      local_message <- paste0(local_message, sprintf(body, c))
    }
  }
}

```

```

} else if(is.nan(tau_hat)) {
  no_warning <- FALSE
  body <- paste0(' Chain %s: The calculation of hat{tau} is ',
                 'unreliable because\nthe empirical ',
                 'autocorrelations did not decay sufficiently ',
                 'quickly.\n')
  local_message <- paste0(local_message, sprintf(body, c))
} else {
  inc_tau_hat <- tau_hat / S
  if (inc_tau_hat > 5) {
    no_warning <- FALSE
    body <- paste0(' Chain %s: Incremental hat{tau} (%.3f) is ',
                   'too large.\n')
    local_message <- paste0(local_message,
                           sprintf(body, c, inc_tau_hat))
  }
}
}
if (no_warning) {
  desc <- paste0('The incremental empirical integrated ',
                 'autocorrelation time is sufficiently well-behaved ',
                 'for the empirical autocorrelation estimates to be ',
                 'reliable.')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, desc, '\n')
} else {
  desc <- paste0('If the incremental empirical integrated ',
                 'autocorrelation times are unreliable or too large ',
                 'then the Markov chains have not explored long ',
                 'enough for the autocorrelation estimates to be ',
                 'reliable.')
  desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
  message <- paste0(message, desc, '\n')
}

cat(message)
}

```

The integrated autocorrelation times moderates the asymptotic variance of well-behaved Markov chain Monte Carlo estimators through the effective sample size,

$$\text{ESS}[f] = \frac{N}{\tau[f]},$$

or in practice the empirical effective sample size that we estimate from the realized Markov chains,

$$\text{ESS}[\hat{f}] = \frac{N}{\hat{\tau}[\hat{f}]}.$$

The effective sample size can be interpreted as how large of an ensemble of exact samples we would need to achieve the same estimator error for the particular expectand of interest.

```
# Compute the minimum empirical effective sample size, or \hat{ESS},
# across the Markov chains for the given expectands.
# @param expectand_vals_list A named list of two-dimensional arrays for
#                             each expectand. The first dimension of
#                             each element indexes the Markov chains and
#                             the second dimension indexes the sequential
#                             states within each Markov chain.
compute_min_ess_hats <- function(expectand_vals_list) {
  validate_named_list_of_arrays(expectand_vals_list,
                                'expectand_vals_list')

  min_ess_hats <- c()
  for (name in names(expectand_vals_list)) {
    expectand_vals <- expectand_vals_list[[name]]
    C <- dim(expectand_vals)[1]
    S <- dim(expectand_vals)[2]

    ess_hats <- rep(0, C)
    for (c in 1:C) {
      tau_hat <- compute_tau_hat(expectand_vals[c,])
      ess_hats[c] <- S / tau_hat
    }
    min_ess_hats <- c(min_ess_hats, min(ess_hats))
  }
  return(min_ess_hats)
}
```

Assuming stationarity we can use the empirical effective sample size to estimate the Markov chain Monte Carlo standard error for any well-behaved expectand estimator

$$\hat{f} \approx \mathbb{E}_{\pi}[f].$$

The necessary effective sample size depends on the precision required for a given Markov chain Monte Carlo estimator. This can vary not only from analysis to analysis but also between multiple expectands within a single analysis. That said an effective sample size of 100 is more than sufficient for most applications and provides a useful rule of thumb. In some applications

even smaller effective sample sizes can yield sufficiently precise Markov chain Monte Carlo estimators.

```
# Check the empirical effective sample size  $\hat{\text{ESS}}$  for the given
# expectand values.
# @param expectand_vals A two-dimensional array of expectand values with
#                         the first dimension indexing the Markov chains
#                         and the second dimension indexing the sequential
#                         states within each Markov chain.
# @param min_ess_hat_per_chain The minimum empirical effective sample
#                               size before a warning message is passed.
# @param max_width Maximum line width for printing
check_ess_hat <- function(expectand_vals,
                          min_ess_hat_per_chain=100,
                          max_width=72) {
  validate_array(expectand_vals, 'expectand_vals')

  C <- dim(expectand_vals)[1]
  S <- dim(expectand_vals)[2]

  no_warning <- TRUE
  message <- ""

  for (c in 1:C) {
    tau_hat <- compute_tau_hat(expectand_vals[c,])
    ess_hat <- S / tau_hat
    if (ess_hat < min_ess_hat_per_chain) {
      body <- paste0(' Chain %s: The empirical effective sample size',
                     '%.3f is too small.\n')
      message <- paste0(message, sprintf(body, c, ess_hat))
      no_warning <- FALSE
    }
  }
  if (no_warning) {
    desc <- paste0('Assuming that a central limit theorem holds the ',
                  'empirical effective sample size is large enough ',
                  'for Markov chain Monte Carlo estimation to be',
                  'reasonably precise.\n\n')
    desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
    message <- paste0(message, desc)
  } else {
    desc <- paste0('Small empirical effective sample sizes result in ',
                  'imprecise Markov chain Monte Carlo estimators.\n\n')
  }
}
```

```

    desc <- paste0(strwrap(desc, max_width, 2), collapse='\n')
    message <- paste0(message, desc)
  }

  cat(message)
}

```

For example empirical effective sample sizes can provide a useful way to distinguish if some diagnostic failures are due to Markov chains that are just too short or more persistent problems.

3.5 All Expectand Diagnostics

In practice we have no reason not to check all of these diagnostics at once for each expectand of interest.

```

# Check all expectand-specific diagnostics.
# @param expectand_vals_list A named list of two-dimensional arrays for
#                             each expectand. The first dimension of
#                             each element indexes the Markov chains and
#                             the second dimension indexes the sequential
#                             states within each Markov chain.
# @param min_ess_hat_per_chain The minimum empirical effective sample
#                             size before a warning message is passed.
# @param exclude_zvar Binary variable to exclude all expectands with
#                             vanishing empirical variance from other diagnostic
#                             checks.
# @param max_width Maximum line width for printing
check_all_expectand_diagnostics <- function(expectand_vals_list,
                                             min_ess_hat_per_chain=100,
                                             exclude_zvar=FALSE,
                                             max_width=72) {
  validate_named_list_of_arrays(expectand_vals_list,
                                'expectand_vals_list')

  no_xi_hat_warning <- TRUE
  no_zvar_warning <- TRUE
  no_rhat_warning <- TRUE
  no_inc_tau_hat_warning <- TRUE
  no_ess_hat_warning <- TRUE

```

```

message <- ""

for (name in names(expectand_vals_list)) {
  if (is.null(expectand_vals_list[[name]])) {
    cat(sprintf('The values for expectand `%s` are ill-formed.',
              name))
    next
  }

  expectand_vals <- expectand_vals_list[[name]]
  C <- dim(expectand_vals)[1]
  S <- dim(expectand_vals)[2]

  local_warning <- FALSE
  local_message <- paste0(name, ':\n')

  if (exclude_zvar) {
    # Check zero variance across all Markov chains for exclusion
    any_zvar <- FALSE
    for (c in 1:C) {
      var <- welford_summary(expectand_vals[c,])[2]
      if (var < 1e-10)
        any_zvar <- TRUE
    }
    if (any_zvar) {
      next
    }
  }

  for (c in 1:C) {
    vals <- expectand_vals[c,]

    # Check tail behavior in each Markov chain
    xi_hat_threshold <- 0.25
    xi_hats <- compute_tail_xi_hats(vals)
    if ( is.nan(xi_hats[1]) & is.nan(xi_hats[2]) ) {
      no_xi_hat_warning <- FALSE
      local_warning <- TRUE
      body <- ' Chain %s: Both left and right hat{xi}s are NaN.\n'
      local_message <- paste0(local_message, sprintf(body, c))
    }
    else if ( is.nan(xi_hats[1]) ) {

```

```

no_xi_hat_warning <- FALSE
local_warning <- TRUE
body <- ' Chain %s: Left hat{xi} is NaN.\n'
local_message <- paste0(local_message, sprintf(body, c))
} else if ( is.nan(xi_hats[2]) ) {
no_xi_hat_warning <- FALSE
local_warning <- TRUE
body <- ' Chain %s: Right hat{xi} is NaN.\n'
local_message <- paste0(local_message, sprintf(body, c))
} else if (xi_hats[1] >= xi_hat_threshold &
xi_hats[2] >= xi_hat_threshold) {
no_xi_hat_warning <- FALSE
local_warning <- TRUE
body <- paste0(' Chain %s: Both left and right tail hat{xi}s ',
'(%f, %f) exceed %f.\n')
local_message <- paste0(local_message,
sprintf(body, c,
xi_hats[1], xi_hats[2],
xi_hat_threshold))
} else if (xi_hats[1] < xi_hat_threshold &
xi_hats[2] >= xi_hat_threshold) {
no_xi_hat_warning <- FALSE
local_warning <- TRUE
body <- ' Chain %s: Right tail hat{xi} (%f) exceeds %f.\n'
local_message <- paste0(local_message,
sprintf(body, c,
xi_hats[2],
xi_hat_threshold))
} else if (xi_hats[1] >= xi_hat_threshold &
xi_hats[2] < xi_hat_threshold) {
no_xi_hat_warning <- FALSE
local_warning <- TRUE
body <- ' Chain %s: Left tail hat{xi} (%f) exceeds %f.\n'
local_message <- paste0(local_message,
sprintf(body, c,
xi_hats[1],
xi_hat_threshold))
}

# Check empirical variance in each Markov chain
var <- welford_summary(vals)[2]
if (var < 1e-10) {

```

```

    no_zvar_warning <- FALSE
    local_warning <- TRUE
    body <- ' Chain %s: Empirical variance is effectively zero.\n'
    local_message <- paste0(local_message, sprintf(body, c))
  }
}

# Check split Rhat across Markov chains
rhat <- compute_split_rhat(expectand_vals)

if (is.nan(rhat)) {
  local_message <- paste0(local_message,
    ' Split hat{R} is ill-defined.\n')
} else if (rhat > 1.1) {
  no_rhat_warning <- FALSE
  local_warning <- TRUE
  body <- ' Split hat{R} (%.3f) exceeds 1.1.\n'
  local_message <- paste0(local_message, sprintf(body, rhat))
}

for (c in 1:C) {
  vals <- expectand_vals[c,]
  tau_hat <- compute_tau_hat(vals)

  if (is.infinite(tau_hat)) {
    no_inc_tau_hat_warning <- FALSE
    local_warning <- TRUE
    body <- paste0(' Chain %s: The calculation of hat{tau} is ',
      'unreliable because\nthe empirical variance ',
      'is zero.\n')
    local_message <- paste0(local_message, sprintf(body, c))
  } else if (is.nan(tau_hat)) {
    no_inc_tau_hat_warning <- FALSE
    local_warning <- TRUE
    body <- paste0(' Chain %s: The calculation of hat{tau} is ',
      'unreliable because\nthe empirical ',
      'autocorrelations did not decay sufficiently ',
      'quickly.\n')
    local_message <- paste0(local_message, sprintf(body, c))
  } else {
    inc_tau_hat <- tau_hat / S
    if (inc_tau_hat > 5) {

```

```

    no_inc_tau_hat_warning <- FALSE
    local_warning <- TRUE
    body <- paste0(' Chain %s: Incremental hat{tau} (%.3f) is ',
                  'too large.\n')
    local_message <- paste0(local_message,
                           sprintf(body, c, inc_tau_hat))
  }
}

# Check empirical effective sample size
ess_hat <- S / tau_hat

if (ess_hat < min_ess_hat_per_chain) {
  no_ess_hat_warning <- FALSE
  local_warning <- TRUE
  body <- paste0(' Chain %s: hat{ESS} (%.3f) is smaller than ',
                'desired (%s).\n')
  local_message <- paste0(local_message,
                         sprintf(body, c, ess_hat,
                                min_ess_hat_per_chain))
}
}

if (local_warning) {
  message <- paste0(message, local_message, '\n')
}
}

if (!no_xi_hat_warning) {
  desc <- paste0('Large tail hat{xi}s suggest that the expectand ',
                'might not be sufficiently integrable.')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, '\n', desc, '\n')
}

if (!no_zvar_warning) {
  desc <- paste0('If the expectands are not constant then zero ',
                'empirical variance suggests that the Markov ',
                'transitions may be misbehaving.')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, '\n', desc, '\n')
}

if (!no_rhat_warning) {

```

```

    desc <- paste0('Split Rhat larger than 1.1 suggests that at ',
                  'least one of the Markov chains has not reached ',
                  'an equilibrium.')
    desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
    message <- paste0(message, '\n', desc, '\n')
  }
  if (!no_inc_tau_hat_warning) {
    desc <- paste0('If the incremental empirical integrated ',
                  'autocorrelation times are unreliable or too large ',
                  'then the Markov chains have not explored long ',
                  'enough for the autocorrelation estimates to be ',
                  'reliable.')
    desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
    message <- paste0(message, '\n', desc, '\n')
  }
  if (!no_ess_hat_warning) {
    desc <- paste0('Small empirical effective sample sizes result in ',
                  'imprecise Markov chain Monte Carlo estimators.')
    desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
    message <- paste0(message, '\n', desc, '\n')
  }

  if(no_xi_hat_warning & no_zvar_warning &
     no_rhat_warning & no_inc_tau_hat_warning & no_ess_hat_warning) {
    desc <- paste0('All expectands checked appear to be behaving ',
                  'well enough for reliable Markov chain Monte ',
                  'Carlo estimation.')
    desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
    message <- paste0(message, desc)
  }

  cat(message)
}

```

That said for particularly problematic fits the output from checking all of the expectands can be overwhelming. In cases where that may be a risk we can summarize the output more compactly.

```

# Summarize all expectand-specific diagnostics.
# @param expectand_vals_list A named list of two-dimensional arrays for
#                             each expectand. The first dimension of
#                             each element indexes the Markov chains and

```

```

#             the second dimension indexes the sequential
#             states within each Markov chain.
# @param min_ess_hat_per_chain The minimum empirical effective sample
#             size before a warning message is passed.
# @param exclude_zvar Binary variable to exclude all expectands with
#             vanishing empirical variance from other diagnostic
#             checks.
# @param max_width Maximum line width for printing
summarize_expectand_diagnostics <- function(expectand_vals_list,
                                           min_ess_hat_per_chain=100,
                                           exclude_zvar=FALSE,
                                           max_width=72) {
  validate_named_list_of_arrays(expectand_vals_list,
                                'expectand_vals_list')

  failed_names <- c()
  failed_xi_hat_names <- c()
  failed_zvar_names <- c()
  failed_rhat_names <- c()
  failed_inc_tau_hat_names <- c()
  failed_ess_hat_names <- c()

  for (name in names(expectand_vals_list)) {
    if (is.null(expectand_vals_list[[name]])) {
      cat(sprintf('The values for expectand `%s` are ill-formed.',
                  name))
      next
    }

    expectand_vals <- expectand_vals_list[[name]]
    C <- dim(expectand_vals)[1]
    S <- dim(expectand_vals)[2]

    if (exclude_zvar) {
      # Check zero variance across all Markov chains for exclusion
      any_zvar <- FALSE
      for (c in 1:C) {
        var <- welford_summary(expectand_vals[c,])[2]
        if (var < 1e-10)
          any_zvar <- TRUE
      }
      if (any_zvar) {

```



```

    next
  }
}

for (c in 1:C) {
  vals <- expectand_vals[c,]

  # Check tail behavior in each Markov chain
  xi_hat_threshold <- 0.25
  xi_hats <- compute_tail_xi_hats(vals)
  if ( is.nan(xi_hats[1]) | is.nan(xi_hats[2]) ) {
    failed_names <- c(failed_names, name)
    failed_xi_hat_nameas <- c(failed_xi_hat_names, name)
  } else if (xi_hats[1] >= xi_hat_threshold |
             xi_hats[2] >= xi_hat_threshold) {
    failed_names <- c(failed_names, name)
    failed_xi_hat_nameas <- c(failed_xi_hat_names, name)
  }

  # Check empirical variance in each Markov chain
  var <- welford_summary(vals)[2]
  if (var < 1e-10) {
    failed_names <- c(failed_names, name)
    failed_zvar_names <- c(failed_zvar_names, name)
  }
}

# Check split Rhats across Markov chains
rhat <- compute_split_rhat(expectand_vals)

if (is.nan(rhat)) {
  failed_names <- c(failed_names, name)
  failed_rhat_names <- c(failed_rhat_names, name)
} else if (rhat > 1.1) {
  failed_names <- c(failed_names, name)
  failed_rhat_names <- c(failed_rhat_names, name)
}

for (c in 1:C) {
  tau_hat <- compute_tau_hat(expectand_vals[c,])

  # Check incremental empirical integrated autocorrelation time

```

```

    if (is.infinite(tau_hat) |
        is.nan(tau_hat)      |
        (tau_hat / S) > 5     ) {
      failed_names <- c(failed_names, name)
      failed_inc_tau_hat_names <- c(failed_inc_tau_hat_names, name)
    }

    # Check empirical effective sample size
    ess_hat <- S / tau_hat

    if (ess_hat < min_ess_hat_per_chain) {
      failed_names <- c(failed_names, name)
      failed_ess_hat_names <- c(failed_ess_hat_names, name)
    }
  }
}

message <- ""

failed_names <- unique(failed_names)
if (length(failed_names)) {
  desc <-
    sprintf('The expectands %s triggered diagnostic warnings.',
            paste(failed_names, collapse=" "))
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, desc, '\n\n')
} else {
  desc <- paste0('All expectands checked appear to be behaving ',
                'well enough for reliable Markov chain Monte ',
                'Carlo estimation.')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, desc, '\n\n')
}

failed_xi_hat_names <- unique(failed_xi_hat_names)
if (length(failed_xi_hat_names)) {
  desc <-
    paste0(sprintf('The expectands %s triggered hat{xi} warnings.\n\n',
                  paste(failed_xi_hat_names, collapse=" ")),
          ' Large tail hat{xi}s suggest that the expectand ',
          'might not be sufficiently integrable.')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
}

```

```

    message <- paste0(message, desc, '\n\n')
  }

failed_zvar_names <- unique(failed_zvar_names)
if (length(failed_zvar_names)) {
  desc <-
    paste0(sprintf('The expectands %s triggered zero variance warnings.\n\n',
      paste(failed_zvar_names, collapse=", "),
      ' If the expectands are not constant then zero ',
      'empirical variance suggests that the Markov ',
      'transitions may be misbehaving.\n\n')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, desc, '')
}

failed_rhat_names <- unique(failed_rhat_names)
if (length(failed_rhat_names)) {
  desc <-
    paste0(sprintf('The expectands %s triggered hat{R} warnings.\n\n',
      paste(failed_rhat_names, collapse=", "),
      ' Split Rhat larger than 1.1 suggests that at ',
      'least one of the Markov chains has not reached ',
      'an equilibrium.')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, desc, '\n\n')
}

failed_inc_tau_hat_names <- unique(failed_inc_tau_hat_names)
if (length(failed_inc_tau_hat_names)) {
  desc <-
    body <- paste0('The expectands %s triggered incremental ',
      'hat{tau} warnings.\n\n')
    paste0(sprintf(body,
      paste(failed_inc_tau_hat_names, collapse=", "),
      'If the incremental empirical integrated ',
      'autocorrelation times are unreliable or too large ',
      'then the Markov chains have not explored long ',
      'enough for the autocorrelation estimates to be ',
      'reliable.')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, desc, '\n\n')
}

```

```

failed_ess_hat_names <- unique(failed_ess_hat_names)
if (length(failed_ess_hat_names)) {
  desc <-
    paste0(sprintf('The expectands %s triggered hat{ESS} warnings.\n\n',
      paste(failed_ess_hat_names, collapse=", "),
      'Small empirical effective sample sizes result in ',
      'imprecise Markov chain Monte Carlo estimators.')
  desc <- paste0(strwrap(desc, max_width, 0), collapse='\n')
  message <- paste0(message, desc, '\n\n')
}

cat(message)
}

```

Alternatively we might filter the expectands, keeping only those of immediate interest.

```

# Filter `expectand_vals_list` object by name.
# @param expectand_vals_list A named list of two-dimensional arrays for
#                             each expectand. The first dimension of
#                             each element indexes the Markov chains and
#                             the second dimension indexes the sequential
#                             states within each Markov chain.
# @param requested_names Expectand names to keep.
# @param check_arrays Binary variable indicating whether or not
#                       requested
#                       names should be expanded to array components.
# @param max_width Maximum line width for printing
# @return A named list of two-dimensional arrays for each requested
#         expectand.
filter_expectands <- function(expectand_vals_list, requested_names,
                              check_arrays=FALSE, max_width=72) {
  validate_named_list_of_arrays(expectand_vals_list,
                                'expectand_vals_list')

  if (length(requested_names) == 0) {
    stop('Input variable requested_names must be non-empty.')
  }

  if (check_arrays == TRUE) {
    good_names <- c()
    bad_names <- c()
  }

```

```

for (name in requested_names) {
  # Search for array suffix
  array_names <- grep(paste0('^', name, '\\['),
                      names(expectand_vals_list),
                      value=TRUE)

  # Append array names, if found
  if (length(array_names) > 0) {
    good_names <- c(good_names, array_names)
  } else {
    if (name %in% names(expectand_vals_list)) {
      # Append bare name, if found
      good_names <- c(good_names, name)
    } else {
      # Add to list of bad names
      bad_names <- c(bad_names, name)
    }
  }
}
} else {
  bad_names <- setdiff(requested_names, names(expectand_vals_list))
  good_names <- intersect(requested_names, names(expectand_vals_list))
}

if (length(bad_names) == 1) {
  message <- paste0(sprintf('The expectand %s ',
                           paste(bad_names, collapse=", ")),
                    'was not found in the expectand_vals_list ',
                    'object and will be ignored.\n\n')
  message <- paste0(strwrap(message, max_width, 0), collapse='\n')
  cat(message)
} else if (length(bad_names) > 1) {
  message <- paste0(sprintf('The expectands %s ',
                           paste(bad_names, collapse=", ")),
                    'were not found in the expectand_vals_list ',
                    'object and will be ignored.\n\n')
  message <- paste0(strwrap(message, max_width, 0), collapse='\n')
  cat(message)
}

expectand_vals_list[good_names]
}

```

3.6 Empirical Autocorrelation Visualization

If we encounter large empirical integrated autocorrelation times, or small estimated effective sample sizes, then we may want to follow up with the empirical autocorrelations themselves. An empirical correlogram provides a useful visualization of these estimates.

```
# Compute empirical autocorrelations for a given Markov chain sequence
# @param vals A one-dimensional array of sequential expectand values.
# @return A one-dimensional array of empirical autocorrelations at each
#         lag up to the length of the sequence.
compute_rhos <- function(vals) {
  # Compute empirical autocorrelations
  N <- length(vals)
  zs <- vals - mean(vals)

  B <- 2**ceiling(log2(N)) # Next power of 2 after N
  zs_buff <- c(zs, rep(0, B - N))

  Fs <- fft(zs_buff)
  Ss <- Fs * Conj(Fs)
  Rs <- fft(Ss, inverse=TRUE)

  acov_buff <- Re(Rs)

  if (acov_buff[1] == 0) {
    return(Inf)
  }

  rhos <- head(acov_buff, N) / acov_buff[1]

  # Drop last lag if (L + 1) is odd so that the
  # lag pairs are complete
  L <- N
  if ((L + 1) %% 2 == 1) {
    L <- L - 1
  }

  # Number of lag pairs
  P <- (L + 1) / 2

  # Construct asymptotic correlation from initial monotone sequence
  old_pair_sum <- rhos[1] + rhos[2]
  max_L <- N

  for (p in 2:P) {
```

```

    current_pair_sum <- rhos[2 * p - 1] + rhos[2 * p]

    if (current_pair_sum < 0) {
      max_L <- 2 * p
      rhos[(max_L + 1):N] <- 0
      break
    }

    if (current_pair_sum > old_pair_sum) {
      current_pair_sum <- old_pair_sum
      rhos[2 * p - 1] <- 0.5 * old_pair_sum
      rhos[2 * p] <- 0.5 * old_pair_sum
    }

    old_pair_sum <- current_pair_sum
  }
  return(rhos)
}

# Plot empirical correlograms for a given expectand across a Markov
# chain ensemble.
# @param expectand_vals A two-dimensional array of expectand values with
#                         the first dimension indexing the Markov chains
#                         and the second dimension indexing the sequential
#                         states within each Markov chain.
# @param max_L Maximum autocorrelation lag
# @param rho_lim Plotting range of autocorrelation values
# @display_name Name of expectand
plot_empirical_correlogram <- function(expectand_vals,
                                       max_L,
                                       rho_lim=c(-0.2, 1.1),
                                       display_name="") {
  validate_array(expectand_vals, 'expectand_vals')
  C <- dim(expectand_vals)[1]

  idx <- rep(0:max_L, each=2)
  xs <- sapply(1:length(idx), function(b) if(b %% 2 == 0) idx[b] + 0.5
                                                    else idx[b] - 0.5)

  plot(0, type="n", main=display_name,
       xlab="Lag", xlim=c(-0.5, max_L + 0.5),
       ylab="Empirical Autocorrelation", ylim=rho_lim)

```

```

abline(h=0, col="#DDDDDD", lty=2, lwd=2)

colors <- c(c_dark, c_mid_highlight, c_mid, c_light_highlight)
for (c in 1:C) {
  rhos <- compute_rhos(expectand_vals[c,])
  pad_rhos <- unlist(lapply(idx, function(n) rhos[n + 1]))
  lines(xs, pad_rhos, lwd=2, col=colors[c])
}
}

```

3.7 Chain-Separated Pairs Plot

We can also visualize strong autocorrelations by coloring the states of each Markov chain in a continuous gradient. When neighboring states are strongly correlated these colors will appear to vary smoothly across the ambient space. More productive Markov transitions result in a more chaotic spray of colors.

```

# Visualize the projection of a Markov chain ensemble along two
# expectands as a pairs plot. Point colors darken along each Markov
# chain to visualize the autocorrelation.
# @param expectand1_vals A two-dimensional array of expectand values
#                        with the first dimension indexing the Markov
#                        chains and the second dimension indexing the
#                        sequential states within each Markov chain.
# @params display_name1 Name of first expectand
# @param expectand2_vals A two-dimensional array of expectand values
#                        with the first dimension indexing the Markov
#                        chains and the second dimension indexing the
#                        sequential states within each Markov chain.
# @params display_name2 Name of second expectand
plot_pairs_by_chain <- function(expectand1_vals, display_name1,
                                expectand2_vals, display_name2) {
  validate_array(expectand1_vals, 'expectand1_vals')
  C1 <- dim(expectand1_vals)[1]
  S1 <- dim(expectand1_vals)[2]

  validate_array(expectand2_vals, 'expectand2_vals')
  C2 <- dim(expectand2_vals)[1]
  S2 <- dim(expectand2_vals)[2]

  if (C1 != C2) {

```



```

C <- min(C1, C2)
C1 <- C
C2 <- C
cat(sprintf('Plotting only %s Markov chains.\n', C))
}

nom_colors <- c("#DCBCBC", "#C79999", "#B97C7C",
               "#A25050", "#8F2727", "#7C0000")
cmap <- colormap(colormap=nom_colors, nshades=max(S1, S2))

min_x <- min(sapply(1:C1, function(c) min(expectand1_vals[c,])))
max_x <- max(sapply(1:C1, function(c) max(expectand1_vals[c,])))

min_y <- min(sapply(1:C2, function(c) min(expectand2_vals[c,])))
max_y <- max(sapply(1:C2, function(c) max(expectand2_vals[c,])))

par(mfrow=c(2, 2), mar = c(5, 5, 3, 1))

for (c in 1:C1) {
  plot(0, type="n", main=paste("Chain", c),
       xlab=display_name1, xlim=c(min_x, max_x),
       ylab=display_name2, ylim=c(min_y, max_y))

  points(unlist(lapply(1:C1, function(c) expectand1_vals[c,])),
         unlist(lapply(1:C2, function(c) expectand2_vals[c,])),
         col="#DDDDDD", pch=16, cex=1.0)
  points(expectand1_vals[c,], expectand2_vals[c,],
         col=cmap, pch=16, cex=1.0)
}
}

```

4 Markov Chain Monte Carlo Estimation

If none of the diagnostics indicate an obstruction to a Markov chain Monte Carlo central limit theorem then we can construct expectation value estimates and their standard errors.

When interested in expectands that have not already been computed we will need to evaluate the existing samples on these new functions, generating pushforward samples. This is particularly straightforward for functions with a single input variable.

```

# Evaluate an expectand on the values of a one-dimensional input
# variable.
# @param input_vals A two-dimensional array of expectand values with
#                   the first dimension indexing the Markov chains
#                   and the second dimension indexing the sequential
#                   states within each Markov chain.
# @param expectand Expectand with one-dimensional input space.
# @return A two-dimensional array of expectand values with the
#         first dimension indexing the Markov chains and the
#         second dimension indexing the sequential states within
#         each Markov chain.
eval_uni_expectand_pushforward <- function(input_vals, expectand) {
  if (dim(input_vals)[1] == 1) {
    as.matrix(t(apply(input_vals, 2, expectand)))
  } else {
    apply(input_vals, 2, expectand)
  }
}

```

Pushing samples forward along more complicated functions is a bit trickier to implement.

```

# Create array of element names from the specified dimensions.
# For example `name_array('x', c(2, 3))` returns the array
# >>      [,1]      [,2]      [,3]
# >> [1,] "x[1,1]" "x[1,2]" "x[1,3]"
# >> [2,] "x[2,1]" "x[2,2]" "x[2,3]"
# @ param base Base name.
# @ param dims Vector of array dimensions.
# @ return Array of element names with dimensions given by dims.
name_array <- function(base, dims) {
  # Validate inputs
  if ( !is.character(base) ) {
    stop(paste0('Input variable ', base, ' is not a character.'))
  }

  if ( !is.vector(dims) | !is.numeric(dims) ) {
    stop(paste0('Input variable ', dims, ' is not a numeric vector.'))
  }

  # Create element names and format them into desired array.
  # Assumes that R arrays are stored in column-major order.
  names <- elem_names(base, dims)
}

```

```

array(names, dims)
}

# Evaluate a list of expectands on the values of an arbitrary number of
# input variables. Expectands must all return a scalar numeric or
# logical output.
#
# By default expectand argument values are accessed by name
# in expectand_vals_list. If a non-null alt_arg_names is provided then
# the alternate names are used to access values in expectand_vals_list.
# The elements of alt_arg_names can also be character arrays of
# arbitrary dimension in which case the individual element values are
# first accessed then formatted into matching numeric arrays before
# being passed to the expectand.
#
# @param expectand_vals_list A named list of two-dimensional arrays.
#                               The first dimension of each element indexes
#                               the Markov chains and the second dimension
#                               indexes the sequential states within each
#                               Markov chain.
# @param expectand_list List of functions with the same arguments.
# @param alt_arg_names Optional named list of alternate names for the
#                       nominal expectand argument names; when used all
#                       expectand argument names must be included.
# @return A list of two-dimensional arrays, one for each element of
#         expectand_list with the same names as expectand_list.
eval_expectand_pushforwards <- function(expectand_vals_list,
                                         expectand_list,
                                         alt_arg_names=NULL) {

  # Validate inputs
  validate_named_list_of_arrays(expectand_vals_list,
                                'expectand_vals_list')

  if (!is.list(expectand_list)) {
    stop(paste0('Input variable `expectand_list` is not a list.'))
  }

  if (!Reduce("&", sapply(expectand_list, is.function))) {
    stop(paste0('The elements of input variable `expectand_list` ',
                'are not all functions.'))
  }
}

```

```

if (!is.null(alt_arg_names)) {
  if ( !is.list(alt_arg_names) |
        is.null(names(alt_arg_names)) ) {
    stop(paste0('Input variable `alt_arg_names` ',
                'is not a named list.'))
  }
}

# Check consistent of expectand arguments
nominal_arg_names <- formalArgs(expectand_list[[1]])

compare_args <- function(e) {
  Reduce("&", formalArgs(e) == nominal_arg_names)
}
arg_consistency <- Reduce("&", sapply(expectand_list, compare_args))
if(!arg_consistency) {
  stop(paste0('The arguments of the functions in `expectand_list` ',
              'are not consistent with each other.'))
}

# Ensure that all argument replacements are arrays
alt_arg_names_array = list()
for (name in names(alt_arg_names)) {
  alt_arg_names_array[[name]] <- as.array(alt_arg_names[[name]])
}

# Check existence of all expectand arguments
if (is.null(alt_arg_names)) {
  check_arg_names <- nominal_arg_names
} else {
  missing_args <- setdiff(nominal_arg_names, names(alt_arg_names))
  if (length(missing_args) == 1) {
    stop(paste0('The nominal expectand argument ',
                paste(missing_args, collapse=", "),
                ' does not have a replacement in ',
                '`alt_arg_names`.'))
  } else if (length(missing_args) > 1) {
    stop(paste0('The nominal expectand arguments ',
                paste(missing_args, collapse=", "),
                ' do not have replacements in ',
                '`alt_arg_names`.'))
  }
}

```

```

check_arg_names <- c(sapply(alt_arg_names,
                           function(alt) as.list(alt)),
                    recursive=TRUE)
}

missing_args <- setdiff(check_arg_names, names(expectand_vals_list))
if (length(missing_args) == 1) {
  stop(paste0('The expectand argument ',
              paste(missing_args, collapse=", "),
              ' is not in `expectand_vals_list`.'))
} else if (length(missing_args)) {
  stop(paste0('The expectand arguments ',
              paste(missing_args, collapse=", "),
              ' are not in `expectand_vals_list`.'))
}

# Apply expectand to all inputs
C <- dim(expectand_vals_list[[1]])[1]
S <- dim(expectand_vals_list[[1]])[2]
pushforward_vals <- lapply(expectand_list,
                           function(e) matrix(NA, nrow=C, ncol=S))

if (!is.null(alt_arg_names))
  alt_names <- lapply(nominal_arg_names,
                     function(name) alt_arg_names_array[[name]])

expectand_vals_env <- as.environment(expectand_vals_list)
access_val <- function(name) {
  expectand_vals_env[[name]][c, s]
}

for (c in 1:C) {
  for (s in 1:S) {
    if (is.null(alt_arg_names)) {
      arg_vals <- lapply(nominal_arg_names, access_val)
    } else {
      arg_vals <- vector("list", length=length(alt_names))
      for (n in seq_along(arg_vals)) {
        arg_vals[[n]] <- apply(alt_names[[n]],
                              seq_along(dim(alt_names[[n]])),
                              access_val)
      }
    }
  }
}

```

```

    }
    for (i in seq_along(pushforward_vals)) {
      pushforward_vals[[i]][c, s] <-
        as.numeric(do.call(expectand_list[[i]], arg_vals))
    }
  }
}

return(pushforward_vals)
}

```

```

# Evaluate an expectand on the values of an arbitrary number of input
# variables. Expectand must return a scalar numeric or logical output.
#
# By default expectand argument values are accessed by name
# in expectand_vals_list. If a non-null alt_arg_names is provided then
# the alternate names are used to access values in expectand_vals_list.
# The elements of alt_arg_names can also be character arrays of
# arbitrary dimension in which case the individual element values are
# first accessed then formatted into matching numeric arrays before
# being passed to the expectand.
#
# @param expectand_vals_list A named list of two-dimensional arrays.
#                               The first dimension of each element indexes
#                               the Markov chains and the second dimension
#                               indexes the sequential states within each
#                               Markov chain.
# @param expectand Function with arbitrary number of scalar and array
#                   input arguments.
# @param alt_arg_names Optional named list of alternate names for the
#                       nominal expectand argument names; when used all
#                       expectand argument names must be included.
# @return A two-dimensional array.
eval_expectand_pushforward <- function(expectand_vals_list,
                                       expectand,
                                       alt_arg_names=NULL) {
  pushforward_vals <- eval_expectand_pushforwards(expectand_vals_list,
                                                  list(expectand),
                                                  alt_arg_names)

  pushforward_vals[[1]]
}

```

Regardless of whether the expectand samples were generated by **Stan** or if we had to derive them ourselves the Markov chain Monte Carlo estimation is the same. In particular we can estimate expectation values using either a single Markov chain or an entire ensemble of Markov chains.

```
# Estimate expectand expectation value from a single Markov chain.
# @param vals A one-dimensional array of sequential expectand values.
# @return The Markov chain Monte Carlo estimate, its estimated standard
#         error, and empirical effective sample size.
mcmc_est <- function(vals) {
  S <- length(vals)
  if (S == 1) {
    return(c(vals[1], 0, NaN))
  }

  summary <- welford_summary(vals)

  if (summary[2] == 0) {
    return(c(summary[1], 0, NaN))
  }

  tau_hat <- compute_tau_hat(vals)
  ess_hat <- S / tau_hat
  return(c(summary[1], sqrt(summary[2] / ess_hat), ess_hat))
}
```

```
# Estimate expectand expectation value from a Markov chain ensemble.
# @param expectand_vals A two-dimensional array of expectand values with
#                       the first dimension indexing the Markov chains
#                       and the second dimension indexing the sequential
#                       states within each Markov chain.
# @return The ensemble Markov chain Monte Carlo estimate, its estimated
#         standard error, and empirical effective sample size.
ensemble_mcmc_est <- function(expectand_vals) {
  validate_array(expectand_vals, 'expectand_vals')

  C <- dim(expectand_vals)[1]
  chain_ests <- lapply(1:C, function(c) mcmc_est(expectand_vals[c,]))

  # Total effective sample size
  total_ess <- sum(sapply(chain_ests, function(est) est[3]))

  if (is.nan(total_ess)) {
```

```

    m <- mean(sapply(chain_est, function(est) est[1]))
    se <- mean(sapply(chain_est, function(est) est[2]))
    return (c(m, se, NaN))
}

# Ensemble average weighted by effective sample size
mean <- sum(sapply(chain_est,
                    function(est) est[3] * est[1])) / total_ess

# Ensemble variance weighed by effective sample size
# including correction for the fact that individual Markov chain
# variances are defined relative to the individual mean estimators
# and not the ensemble mean estimator
vars <- rep(0, C)

for (c in 1:C) {
  est <- chain_est[[c]]
  chain_var <- est[3] * est[2]**2
  var_update <- (est[1] - mean)**2
  vars[c] <- est[3] * (var_update + chain_var)
}
var <- sum(vars) / total_ess

c(mean, sqrt(var / total_ess), total_ess)
}

```

A particularly common probabilistic calculation is estimating the probability allocated to subsets that are defined only implicitly by an indicator function. In practice we can estimate this probability by pushing samples forward along the indicator function and then constructing the Markov chain Monte Carlo expectation value estimator. These two steps occur together so often that it's helpful to wrap them into a dedicated function.

```

# Estimate the probability allocated to a subset implicitly defined by
# an indicator function.
# @param expectand_vals_list A named list of two-dimensional arrays.
#                               The first dimension of each element indexes
#                               the Markov chains and the second dimension
#                               indexes the sequential states within each
#                               Markov chain.
# @param indicator Function with logical or 0/1 numeric outputs.
# @param alt_arg_names Optional named list of alternate names for the
#                       nominal expectand argument names; when used all

```



```

#           expectand argument names must be included.
# @return The probability estimate and its standard error.
implicit_subset_prob <- function(expectand_vals_list,
                                indicator,
                                alt_arg_names) {

  # Evaluate indicator function
  indicator_samples <- eval_expectand_pushforward(expectand_vals_list,
                                                  indicator,
                                                  alt_arg_names)

  # Verify outputs
  unique_vals <- unique(c(indicator_samples, recursive=TRUE))

  valid_outputs = TRUE
  if (length(unique_vals) == 1) {
    if (!(unique_vals == 0 | unique_vals == 1))
      valid_outputs = FALSE
  } else if (length(unique_vals) == 2) {
    if ( !( Reduce("&", unique_vals == c(0, 1))
           | Reduce("&", unique_vals == c(1, 0))) )
      valid_outputs = FALSE
  } else {
    valid_outputs = FALSE
  }
  if(!valid_outputs)
    stop(paste0('The function `indicator` must return only ',
                'logical or 0/1 numeric outputs.'))

  # Return probability as expectation value of indicator function
  ensemble_mcmc_est(indicator_samples)[1:2]
}

```

We can also use realized Markov chains to estimate quantiles of the pushforward distribution along an expectand. Within a single Markov chain ordering the expectand values allow us to efficiently search for the value x_q whose corresponding interval probability first exceeds the defining quantile probability p ,

$$\begin{aligned}
p &< \pi(\{-\infty, x_q\}) \\
&= \mathbb{E}_\pi \left[I_{\{-\infty, x_q\}} \right] \\
&\approx \frac{1}{N} \sum_{n=1}^N I_{\{-\infty, x_q\}}(\tilde{x}_n).
\end{aligned}$$

The empirical quantiles within each Markov chain can then be averaged together to provide an ensemble estimator.

In theory the empirical standard deviation of the individual Markov chain estimates consistently estimates the estimator error, but the estimation is unreliable without many Markov chains. Consequently it is not reported here.

```
# Estimate expectand pushforward quantiles from a Markov chain ensemble.
# @param expectand_vals A two-dimensional array of expectand values with
#                       the first dimension indexing the Markov chains
#                       and the second dimension indexing the sequential
#                       states within each Markov chain.
# @param probs An array of quantile probabilities.
# @return The ensemble Markov chain Monte Carlo quantile estimate.
ensemble_mcmc_quantile_est <- function(expectand_vals, probs) {
  # Validate inputs
  validate_array(expectand_vals, 'expectand_vals')

  if (!is.vector(probs)) {
    stop(paste0('Input variable `probs` is not a ',
                'one-dimensional numeric array.'))
  }

  # Estimate and return quantile
  q <- 0 * probs

  C <- dim(expectand_vals)[1]
  for (c in 1:C) {
    q <- q + quantile(expectand_vals[c,], probs=probs) / C
  }

  return(q)
}
```

Finally we can also visualize the entire pushforward distribution by estimating the target probabilities in histogram bins.

```
# Visualize pushforward distribution along a given expectand as a
# sequence of bin probabilities weighted by bin widths that approximates
# the pushforward probability density function. Markov chain Monte
# Carlo estimates the output bin probabilities from the input samples,
# with the bin probability estimator errors visualized in the border
# color.
```

```

# @param expectand_vals A two-dimensional array of expectand values with
#                         the first dimension indexing the Markov chains
#                         and the second dimension indexing the sequential
#                         states within each Markov chain.
# @param B The number of histogram bins
# @param display_name Expectand name
# @param flim Optional histogram range
# @param ylim Optional y-axis range; ignored if add is TRUE
# @param col Color for plotting weighted bin probabilities; defaults to
#             c_dark.
# @param border Color for plotting estimator error; defaults to gray
# @param add Configure plot to overlay over existing plot; defaults to
#             FALSE
# @param main Optional plot title
# @param baseline Optional baseline value for visual comparison
# @param baseline_col Color for plotting baseline value; defaults to
#                     "black"
plot_expectand_pushforward <- function(expectand_vals, B,
                                       display_name="f",
                                       flim=NULL, ylim=NULL,
                                       col=c_dark, border="#DDDDDD",
                                       add=FALSE, main="",
                                       baseline=NULL,
                                       baseline_col="black") {
  validate_array(expectand_vals, 'expectand_vals')

  # Automatically adjust histogram range to range of expectand values
  # if range is not already set as an input variable
  if (is.null(flim)) {
    min_f <- min(expectand_vals)
    max_f <- max(expectand_vals)
    delta <- (max_f - min_f) / B

    # Add bounding bins
    B <- B + 2
    min_f <- min_f - delta
    max_f <- max_f + delta
    flim <- c(min_f, max_f)

    bins <- seq(min_f, max_f, delta)
  } else {
    min_f <- flim[1]

```

```

max_f <- flim[2]

delta <- (max_f - min_f) / B
bins <- seq(min_f, max_f, delta)
}

# Check value containment
S <- dim(expectand_vals)[1] * dim(expectand_vals)[2]

S_low <- sum(c(expectand_vals, recursive=TRUE) < min_f)
if (S_low == 1)
  warning(sprintf('%i value (%.1f%%) fell below the histogram binning.',
                  S_low, 100 * S_low / S))
else if (S_low > 1)
  warning(sprintf('%i values (%.1f%%) fell below the histogram binning.',
                  S_low, 100 * S_low / S))

S_high <- sum(max_f < c(expectand_vals, recursive=TRUE))
if (S_high == 1)
  warning(sprintf('%i value (%.1f%%) fell above the histogram binning.',
                  S_high, 100 * S_high / S))
else if (S_high > 1)
  warning(sprintf('%i values (%.1f%%) fell above the histogram binning.',
                  S_high, 100 * S_high / S))

# Compute bin heights
mean_p <- rep(0, B)
delta_p <- rep(0, B)

for (b in 1:B) {
  # Estimate bin probabilities
  bin_indicator <- function(x) {
    ifelse(bins[b] <= x & x < bins[b + 1], 1, 0)
  }
  indicator_vals <- eval_uni_expectand_pushforward(expectand_vals,
                                                    bin_indicator)
  est <- ensemble_mcmc_est(indicator_vals)

  # Normalize bin probabilities by bin width to allow
  # for direct comparison to probability density functions
  width = bins[b + 1] - bins[b]
  mean_p[b] = est[1] / width
}

```

```

    delta_p[b] = est[2] / width
  }

# Plot histogram
idx <- rep(1:B, each=2)
x <- sapply(1:length(idx), function(b) if(b %% 2 == 1) bins[idx[b]]
           else bins[idx[b] + 1])
lower_inter <- sapply(idx, function(n)
  max(mean_p[n] - 2 * delta_p[n], 0))
upper_inter <- sapply(idx, function(n)
  min(mean_p[n] + 2 * delta_p[n], 1 / width))

if (add) {
  polygon(c(x, rev(x)), c(lower_inter, rev(upper_inter)),
          col=border, border=NA)
  lines(x, mean_p[idx], col=col, lwd=2)
} else {
  if (is.null(ylim)) {
    ylim=c(0, max(1.05 * upper_inter))
  }

  plot(1, type="n", main=main,
        xlim=flim, xlab=display_name,
        ylim=ylim, ylab="", yaxt="n")
  title(ylab="Estimated Bin\nProbabilities / Bin Width",
        mgp=c(1, 1, 0))

  polygon(c(x, rev(x)), c(lower_inter, rev(upper_inter)),
          col=border, border=NA)
  lines(x, mean_p[idx], col=col, lwd=2)
}

# Plot baseline if applicable
if (!is.null(baseline)) {
  abline(v=baseline, col="white", lty=1, lwd=4)
  abline(v=baseline, col=baseline_col, lty=1, lwd=2)
}
}

```

5 Demonstration

Now let's put all of these analysis tools to use with an `rstan` fit object.

First we setup our local R environment.

```
library(rstan)
rstan_options(auto_write = TRUE)          # Cache compiled Stan programs
options(mc.cores = parallel::detectCores()) # Parallelize chains
parallel::setDefaultClusterOptions(setup_strategy = "sequential")
```

Next we source all of these diagnostics into a local environment to avoid any conflicts with other functions.

```
util <- new.env()
source('mcmc_analysis_tools_rstan.R', local=util)
```

Then we can simulate some binary data from a logistic regression model.

```
simu <- stan(file="stan_programs/simu_logistic_reg.stan",
             iter=1, warmup=0, chains=1,
             seed=4838282, algorithm="Fixed_param")
```

```
SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
```

```
Chain 1: Iteration: 1 / 1 [100%] (Sampling)
```

```
Chain 1:
```

```
Chain 1: Elapsed Time: 0 seconds (Warm-up)
```

```
Chain 1:           0 seconds (Sampling)
```

```
Chain 1:           0 seconds (Total)
```

```
Chain 1:
```

```
X <- extract(simu)$X[1,,]
```

```
y <- extract(simu)$y[1,]
```

```
data <- list("M" = 3, "N" = 1000, "x0" = c(-1, 0, 1), "X" = X, "y" = y)
```

We'll try to fit this model not with a constraint-respecting logistic regression model but rather a constraint blaspheming linear probability model. Importantly the resulting posterior density function is discontinuous with configurations $\alpha + \delta X * \beta > 0$ resulting in finite

bernoulli_lpmf outputs and those with $\alpha + \delta X * \beta \leq 0$ resulting in minus infinite outputs.

Because of this awkward constraint we have to carefully initialize our Markov chains to satisfy the $\alpha + \delta X * \beta > 0$ constraint.

```
set.seed(48383499)

interval_inits <- list()

for (c in 1:4) {
  beta <- c(0, 0, 0)
  alpha <- rnorm(1, 0.5, 0.1)
  interval_inits[[c]] <- list("alpha" = alpha, "beta" = beta)
}

fit <- stan(file="stan_programs/bernoulli_linear.stan",
            data=data, seed=8438338,
            warmup=1000, iter=2024, refresh=0,
            init=interval_inits)
```

Stan is able to run to completion, but just how useful are the Markov chains that it generates?

Let's start with the Hamiltonian Monte Carlo diagnostics.

```
diagnostics <- util$extract_hmc_diagnostics(fit)
util$check_all_hmc_diagnostics(diagnostics)
```

Chain 1: 1022 of 1024 transitions (99.8%) diverged.

Chain 2: 1014 of 1024 transitions (99.0%) diverged.

Chain 3: 1015 of 1024 transitions (99.1%) diverged.

Chain 4: 1013 of 1024 transitions (98.9%) diverged.

Chain 4: Average proxy acceptance statistic (0.629)
is smaller than 90% of the target (0.801).

Divergent Hamiltonian transitions result from unstable numerical trajectories. These instabilities are often due to degenerate target geometry, especially "pinches". If there are only a small number of divergences then running with `adapt_delta` larger than 0.801 may reduce the instabilities at the cost of more expensive Hamiltonian

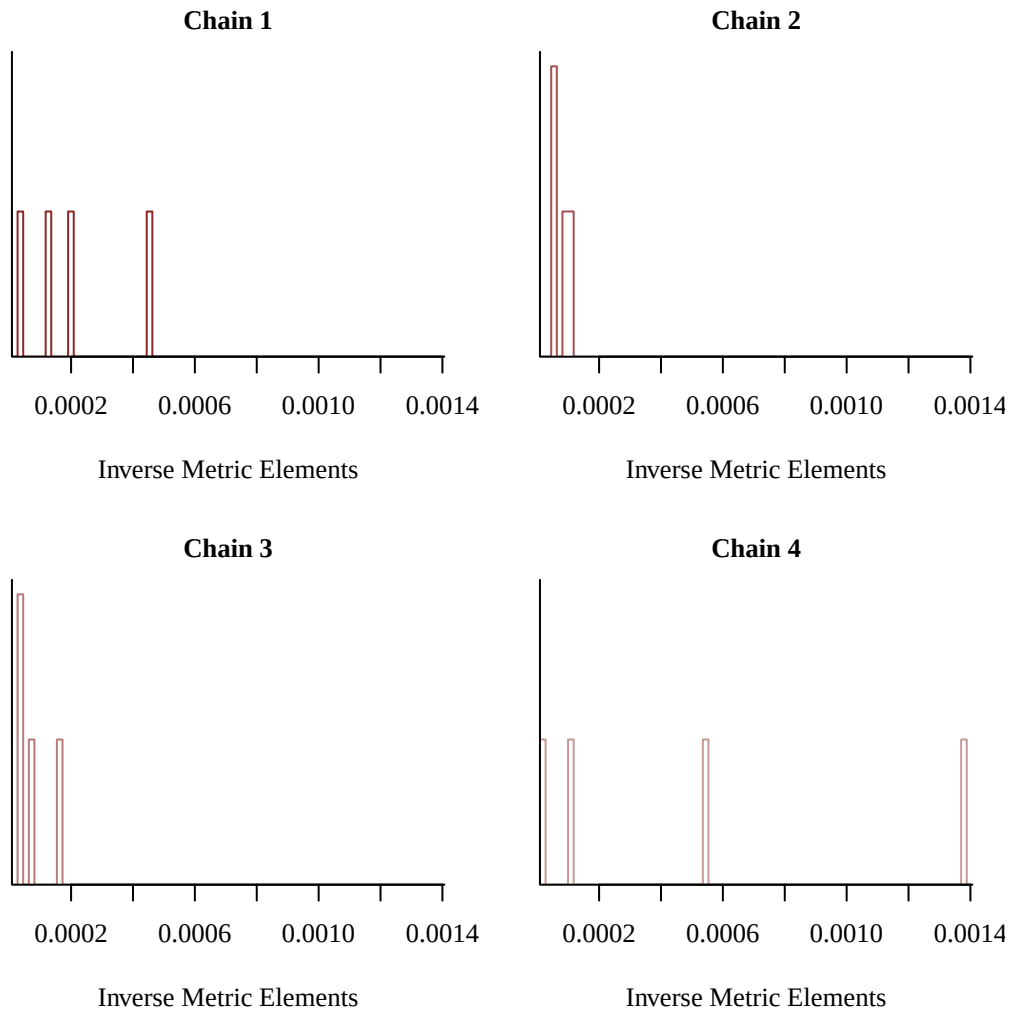
transitions.

A small average proxy acceptance statistic indicates that the adaptation of the numerical integrator step size failed to converge. This is often due to discontinuous or imprecise gradients.

Almost every transition across the four Markov chains resulted in a divergence. This is due to the discontinuity in the linear probability model as the sudden jump from a finite to a negative infinite target density results in unstable numerical trajectories.

We also see the one of the Markov chains wasn't able to hit the step size adaptation target. To see why let's dig into the adapted configuration of the Hamiltonian Markov transition.

```
util$plot_inv_metric(fit, 75)
```



The problematic Markov chain also exhibits the most variation in its inverse metric elements, which in this case is probably an artifact of its warmup phase spending too much time close to a constraint boundary. Artificially variable inverse metric elements frustrate numerical integration which can then frustrate the integrator step size adaptation.

Interestingly the adapted step sizes are nearly the same for all four Markov chains. The lower average proxy acceptance statistic seen in the fourth Markov chain is due entirely to the wonky inverse metric adaptation.

```
util$display_stepsizes(diagnostics)
```

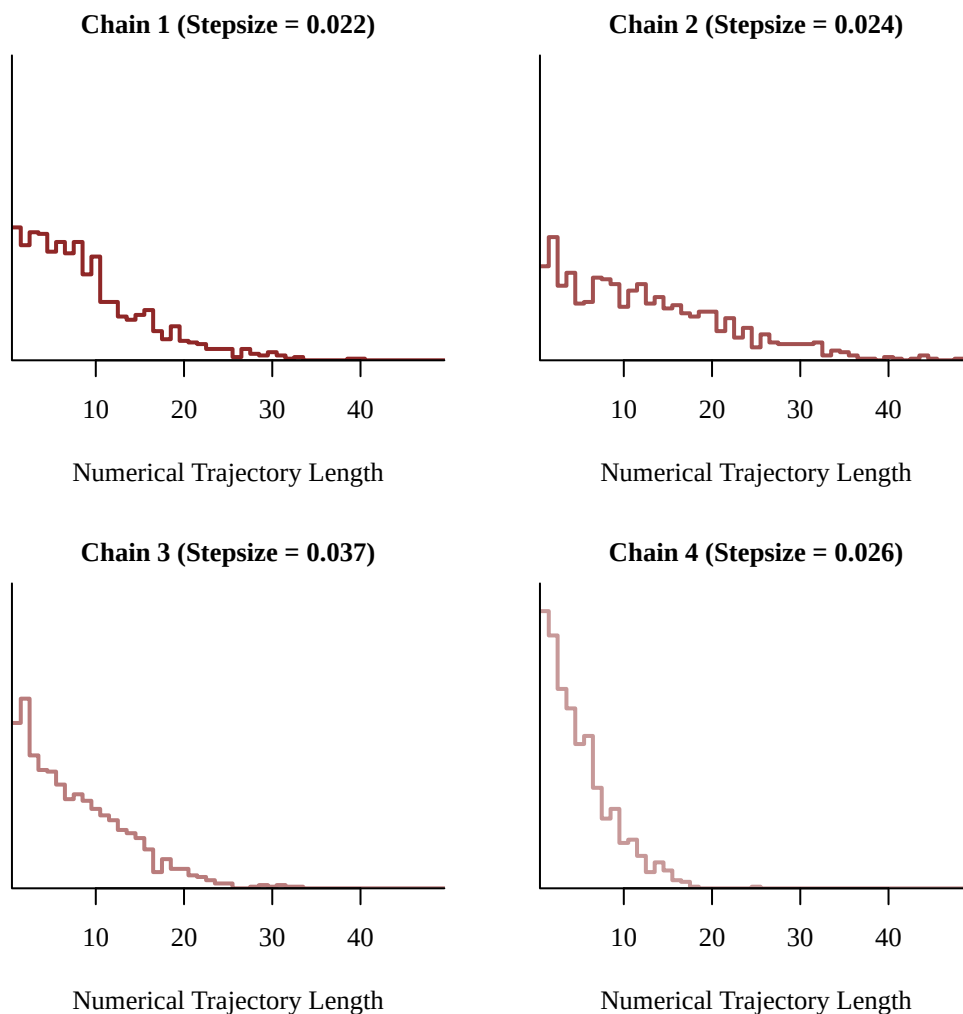
```
Chain 1: Integrator Step Size = 0.022103
Chain 2: Integrator Step Size = 0.024148
Chain 3: Integrator Step Size = 0.037035
Chain 4: Integrator Step Size = 0.026320
```

```
util$display_ave_accept_proxy(diagnostics)
```

```
Chain 1: Average proxy acceptance statistic = 0.766
Chain 2: Average proxy acceptance statistic = 0.814
Chain 3: Average proxy acceptance statistic = 0.729
Chain 4: Average proxy acceptance statistic = 0.629
```

The different inverse metric results in different Hamiltonian dynamics. In this case the dynamics driving the fourth Markov chain are not able to explore as far as those in the other chains.

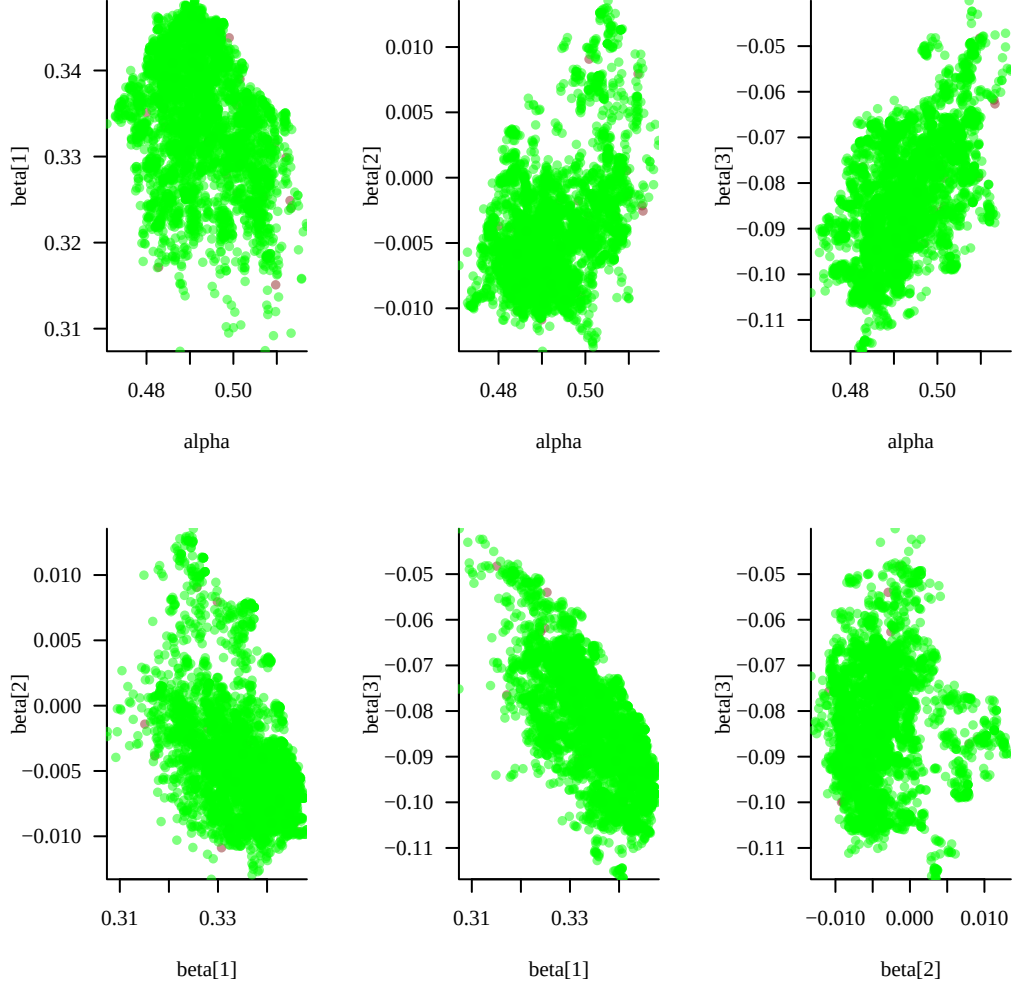
```
util$plot_num_leapfrogs_by_chain(diagnostics)
```



Finally because nearly every transition is divergent we can't extract much information from the divergent-labeled pairs plots.

```
samples <- util$extract_expectand_vals(fit)

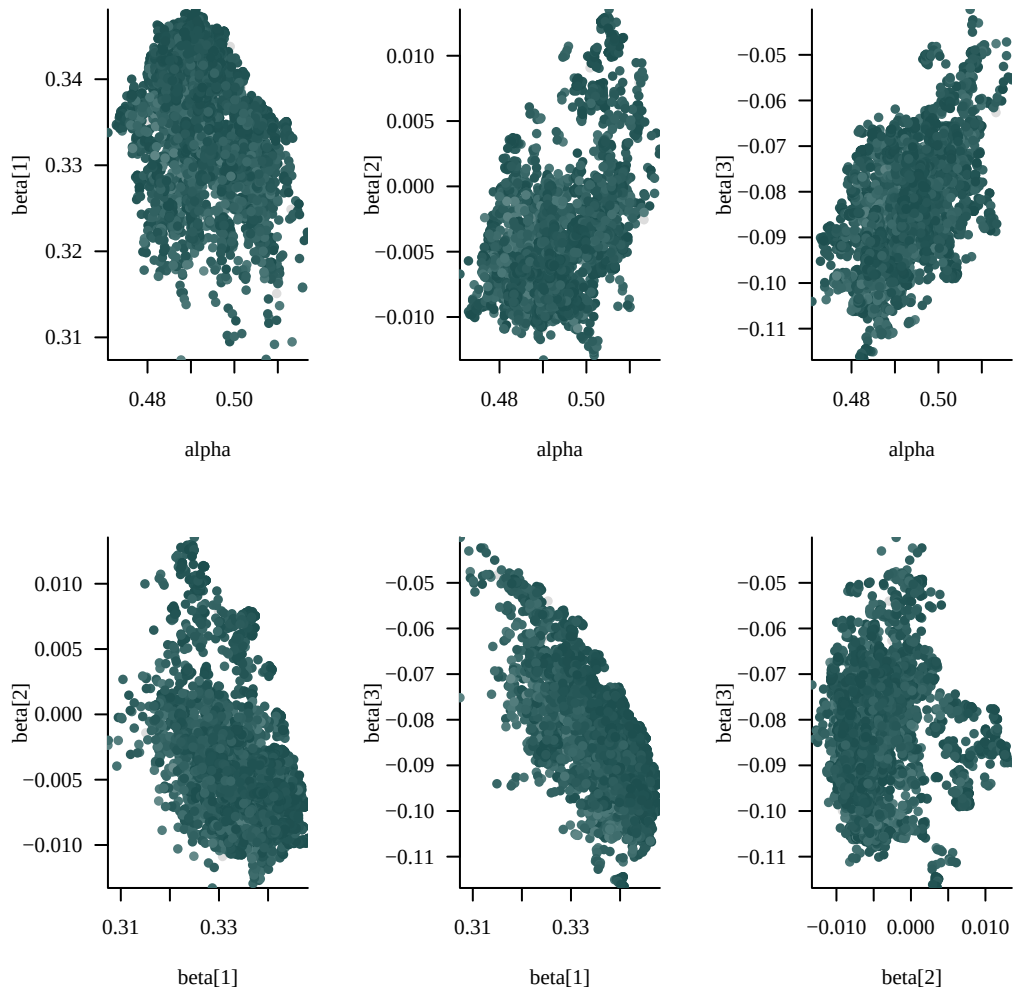
names <- c('alpha',
           sapply(1:data$M, function(m) paste0('beta[', m, ']')))
util$plot_div_pairs(names, names, samples, diagnostics)
```



We can also color the divergent transitions by their numerical trajectory lengths. On average transitions from shorter numerical trajectories should be closer to the problematic behavior than transitions from longer numerical trajectories. Because there are so many divergent transitions here the point colors overlap and it's hard to make too much out, but it does look like there may be a problematic boundary. For example plot of `beta[2]` against `beta[1]` is consistent with a boundary defined by

$$\beta_1 + \beta_2 = \text{constant}.$$

```
util$plot_div_pairs(names, names, samples, diagnostics, plot_mode=1)
```



Having examined the Hamiltonian Monte Carlo diagnostics let's now look through the expectand specific diagnostics. By default we'll look at the parameter projection functions as well as all of the expectands defined in the `generated quantities` block.

Because of the Hamiltonian Monte Carlo diagnostic failures I'm going to limit the output just in case we have many failures for these diagnostics as well.

```
util$summarize_expectand_diagnostics(samples)
```

The expectands `alpha`, `beta[1]`, `beta[2]`, `beta[3]`, `p[1]`, `p[2]`, `p[3]`, `p[4]`, `p[5]`, `p[6]`, `p[7]`, `p[8]`, `p[9]`, `p[10]`, `p[11]`, `p[12]`, `p[13]`, `p[14]`, `p[15]`, `p[16]`, `p[17]`, `p[18]`, `p[19]`, `p[20]`, `p[21]`, `p[22]`, `p[23]`, `p[24]`,

p[25], p[26], p[27], p[28], p[29], p[30], p[31], p[32], p[33], p[34],
p[35], p[36], p[37], p[38], p[39], p[40], p[41], p[42], p[43], p[44],
p[45], p[46], p[47], p[48], p[49], p[50], p[51], p[52], p[53], p[54],
p[55], p[56], p[57], p[58], p[59], p[60], p[61], p[62], p[63], p[64],
p[65], p[66], p[67], p[68], p[69], p[70], p[71], p[72], p[73], p[74],
p[75], p[76], p[77], p[78], p[79], p[80], p[81], p[82], p[83], p[84],
p[85], p[86], p[87], p[88], p[89], p[90], p[91], p[92], p[93], p[94],
p[95], p[96], p[97], p[98], p[99], p[100], p[101], p[102], p[103],
p[104], p[105], p[106], p[107], p[108], p[109], p[110], p[111], p[112],
p[113], p[114], p[115], p[116], p[117], p[118], p[119], p[120], p[121],
p[122], p[123], p[124], p[125], p[126], p[127], p[128], p[129], p[130],
p[131], p[132], p[133], p[134], p[135], p[136], p[137], p[138], p[139],
p[140], p[141], p[142], p[143], p[144], p[145], p[146], p[147], p[148],
p[149], p[150], p[151], p[152], p[153], p[154], p[155], p[156], p[157],
p[158], p[159], p[160], p[161], p[162], p[163], p[164], p[165], p[166],
p[167], p[168], p[169], p[170], p[171], p[172], p[173], p[174], p[175],
p[176], p[177], p[178], p[179], p[180], p[181], p[182], p[183], p[184],
p[185], p[186], p[187], p[188], p[189], p[190], p[191], p[192], p[193],
p[194], p[195], p[196], p[197], p[198], p[199], p[200], p[201], p[202],
p[203], p[204], p[205], p[206], p[207], p[208], p[209], p[210], p[211],
p[212], p[213], p[214], p[215], p[216], p[217], p[218], p[219], p[220],
p[221], p[222], p[223], p[224], p[225], p[226], p[227], p[228], p[229],
p[230], p[231], p[232], p[233], p[234], p[235], p[236], p[237], p[238],
p[239], p[240], p[241], p[242], p[243], p[244], p[245], p[246], p[247],
p[248], p[249], p[250], p[251], p[252], p[253], p[254], p[255], p[256],
p[257], p[258], p[259], p[260], p[261], p[262], p[263], p[264], p[265],
p[266], p[267], p[268], p[269], p[270], p[271], p[272], p[273], p[274],
p[275], p[276], p[277], p[278], p[279], p[280], p[281], p[282], p[283],
p[284], p[285], p[286], p[287], p[288], p[289], p[290], p[291], p[292],
p[293], p[294], p[295], p[296], p[297], p[298], p[299], p[300], p[301],
p[302], p[303], p[304], p[305], p[306], p[307], p[308], p[309], p[310],
p[311], p[312], p[313], p[314], p[315], p[316], p[317], p[318], p[319],
p[320], p[321], p[322], p[323], p[324], p[325], p[326], p[327], p[328],
p[329], p[330], p[331], p[332], p[333], p[334], p[335], p[336], p[337],
p[338], p[339], p[340], p[341], p[342], p[343], p[344], p[345], p[346],
p[347], p[348], p[349], p[350], p[351], p[352], p[353], p[354], p[355],
p[356], p[357], p[358], p[359], p[360], p[361], p[362], p[363], p[364],
p[365], p[366], p[367], p[368], p[369], p[370], p[371], p[372], p[373],
p[374], p[375], p[376], p[377], p[378], p[379], p[380], p[381], p[382],
p[383], p[384], p[385], p[386], p[387], p[388], p[389], p[390], p[391],
p[392], p[393], p[394], p[395], p[396], p[397], p[398], p[399], p[400],
p[401], p[402], p[403], p[404], p[405], p[406], p[407], p[408], p[409],
p[410], p[411], p[412], p[413], p[414], p[415], p[416], p[417], p[418],

p[806], p[807], p[808], p[809], p[810], p[811], p[812], p[813], p[814],
p[815], p[816], p[817], p[818], p[819], p[820], p[821], p[822], p[823],
p[824], p[825], p[826], p[827], p[828], p[829], p[830], p[831], p[832],
p[833], p[834], p[835], p[836], p[837], p[838], p[839], p[840], p[841],
p[842], p[843], p[844], p[845], p[846], p[847], p[848], p[849], p[850],
p[851], p[852], p[853], p[854], p[855], p[856], p[857], p[858], p[859],
p[860], p[861], p[862], p[863], p[864], p[865], p[866], p[867], p[868],
p[869], p[870], p[871], p[872], p[873], p[874], p[875], p[876], p[877],
p[878], p[879], p[880], p[881], p[882], p[883], p[884], p[885], p[886],
p[887], p[888], p[889], p[890], p[891], p[892], p[893], p[894], p[895],
p[896], p[897], p[898], p[899], p[900], p[901], p[902], p[903], p[904],
p[905], p[906], p[907], p[908], p[909], p[910], p[911], p[912], p[913],
p[914], p[915], p[916], p[917], p[918], p[919], p[920], p[921], p[922],
p[923], p[924], p[925], p[926], p[927], p[928], p[929], p[930], p[931],
p[932], p[933], p[934], p[935], p[936], p[937], p[938], p[939], p[940],
p[941], p[942], p[943], p[944], p[945], p[946], p[947], p[948], p[949],
p[950], p[951], p[952], p[953], p[954], p[955], p[956], p[957], p[958],
p[959], p[960], p[961], p[962], p[963], p[964], p[965], p[966], p[967],
p[968], p[969], p[970], p[971], p[972], p[973], p[974], p[975], p[976],
p[977], p[978], p[979], p[980], p[981], p[982], p[983], p[984], p[985],
p[986], p[987], p[988], p[989], p[990], p[991], p[992], p[993], p[994],
p[995], p[996], p[997], p[998], p[999], p[1000], y_pred[1], y_pred[2],
y_pred[3], y_pred[4], y_pred[5], y_pred[6], y_pred[7], y_pred[8],
y_pred[9], y_pred[10], y_pred[11], y_pred[12], y_pred[13], y_pred[14],
y_pred[15], y_pred[16], y_pred[17], y_pred[18], y_pred[19], y_pred[20],
y_pred[21], y_pred[22], y_pred[23], y_pred[24], y_pred[25], y_pred[26],
y_pred[27], y_pred[28], y_pred[29], y_pred[30], y_pred[31], y_pred[32],
y_pred[33], y_pred[34], y_pred[35], y_pred[36], y_pred[37], y_pred[38],
y_pred[39], y_pred[40], y_pred[41], y_pred[42], y_pred[43], y_pred[44],
y_pred[45], y_pred[46], y_pred[47], y_pred[48], y_pred[49], y_pred[50],
y_pred[51], y_pred[52], y_pred[53], y_pred[54], y_pred[55], y_pred[56],
y_pred[57], y_pred[58], y_pred[59], y_pred[60], y_pred[61], y_pred[62],
y_pred[63], y_pred[64], y_pred[65], y_pred[66], y_pred[67], y_pred[68],
y_pred[69], y_pred[70], y_pred[71], y_pred[72], y_pred[73], y_pred[74],
y_pred[75], y_pred[76], y_pred[77], y_pred[78], y_pred[79], y_pred[80],
y_pred[81], y_pred[82], y_pred[83], y_pred[84], y_pred[85], y_pred[86],
y_pred[87], y_pred[88], y_pred[89], y_pred[90], y_pred[91], y_pred[92],
y_pred[93], y_pred[94], y_pred[95], y_pred[96], y_pred[97], y_pred[98],
y_pred[99], y_pred[100], y_pred[101], y_pred[102], y_pred[103],
y_pred[104], y_pred[105], y_pred[106], y_pred[107], y_pred[108],
y_pred[109], y_pred[110], y_pred[111], y_pred[112], y_pred[113],
y_pred[114], y_pred[115], y_pred[116], y_pred[117], y_pred[118],
y_pred[119], y_pred[120], y_pred[121], y_pred[122], y_pred[123],

y_pred[124], y_pred[125], y_pred[126], y_pred[127], y_pred[128],
y_pred[129], y_pred[130], y_pred[131], y_pred[132], y_pred[133],
y_pred[134], y_pred[135], y_pred[136], y_pred[137], y_pred[138],
y_pred[139], y_pred[140], y_pred[141], y_pred[142], y_pred[143],
y_pred[144], y_pred[145], y_pred[146], y_pred[147], y_pred[148],
y_pred[149], y_pred[150], y_pred[151], y_pred[152], y_pred[153],
y_pred[154], y_pred[155], y_pred[156], y_pred[157], y_pred[158],
y_pred[159], y_pred[160], y_pred[161], y_pred[162], y_pred[163],
y_pred[164], y_pred[165], y_pred[166], y_pred[167], y_pred[168],
y_pred[169], y_pred[170], y_pred[171], y_pred[172], y_pred[173],
y_pred[174], y_pred[175], y_pred[176], y_pred[177], y_pred[178],
y_pred[179], y_pred[180], y_pred[181], y_pred[182], y_pred[183],
y_pred[184], y_pred[185], y_pred[186], y_pred[187], y_pred[188],
y_pred[189], y_pred[190], y_pred[191], y_pred[192], y_pred[193],
y_pred[194], y_pred[195], y_pred[196], y_pred[197], y_pred[198],
y_pred[199], y_pred[200], y_pred[201], y_pred[202], y_pred[203],
y_pred[204], y_pred[205], y_pred[206], y_pred[207], y_pred[208],
y_pred[209], y_pred[210], y_pred[211], y_pred[212], y_pred[213],
y_pred[214], y_pred[215], y_pred[216], y_pred[217], y_pred[218],
y_pred[219], y_pred[220], y_pred[221], y_pred[222], y_pred[223],
y_pred[224], y_pred[225], y_pred[226], y_pred[227], y_pred[228],
y_pred[229], y_pred[230], y_pred[231], y_pred[232], y_pred[233],
y_pred[234], y_pred[235], y_pred[236], y_pred[237], y_pred[238],
y_pred[239], y_pred[240], y_pred[241], y_pred[242], y_pred[243],
y_pred[244], y_pred[245], y_pred[246], y_pred[247], y_pred[248],
y_pred[249], y_pred[250], y_pred[251], y_pred[252], y_pred[253],
y_pred[254], y_pred[255], y_pred[256], y_pred[257], y_pred[258],
y_pred[259], y_pred[260], y_pred[261], y_pred[262], y_pred[263],
y_pred[264], y_pred[265], y_pred[266], y_pred[267], y_pred[268],
y_pred[269], y_pred[270], y_pred[271], y_pred[272], y_pred[273],
y_pred[274], y_pred[275], y_pred[276], y_pred[277], y_pred[278],
y_pred[279], y_pred[280], y_pred[281], y_pred[282], y_pred[283],
y_pred[284], y_pred[285], y_pred[286], y_pred[287], y_pred[288],
y_pred[289], y_pred[290], y_pred[291], y_pred[292], y_pred[293],
y_pred[294], y_pred[295], y_pred[296], y_pred[297], y_pred[298],
y_pred[299], y_pred[300], y_pred[301], y_pred[302], y_pred[303],
y_pred[304], y_pred[305], y_pred[306], y_pred[307], y_pred[308],
y_pred[309], y_pred[310], y_pred[311], y_pred[312], y_pred[313],
y_pred[314], y_pred[315], y_pred[316], y_pred[317], y_pred[318],
y_pred[319], y_pred[320], y_pred[321], y_pred[322], y_pred[323],
y_pred[324], y_pred[325], y_pred[326], y_pred[327], y_pred[328],
y_pred[329], y_pred[330], y_pred[331], y_pred[332], y_pred[333],
y_pred[334], y_pred[335], y_pred[336], y_pred[337], y_pred[338],

y_pred[339], y_pred[340], y_pred[341], y_pred[342], y_pred[343],
y_pred[344], y_pred[345], y_pred[346], y_pred[347], y_pred[348],
y_pred[349], y_pred[350], y_pred[351], y_pred[352], y_pred[353],
y_pred[354], y_pred[355], y_pred[356], y_pred[357], y_pred[358],
y_pred[359], y_pred[360], y_pred[361], y_pred[362], y_pred[363],
y_pred[364], y_pred[365], y_pred[366], y_pred[367], y_pred[368],
y_pred[369], y_pred[370], y_pred[371], y_pred[372], y_pred[373],
y_pred[374], y_pred[375], y_pred[376], y_pred[377], y_pred[378],
y_pred[379], y_pred[380], y_pred[381], y_pred[382], y_pred[383],
y_pred[384], y_pred[385], y_pred[386], y_pred[387], y_pred[388],
y_pred[389], y_pred[390], y_pred[391], y_pred[392], y_pred[393],
y_pred[394], y_pred[395], y_pred[396], y_pred[397], y_pred[398],
y_pred[399], y_pred[400], y_pred[401], y_pred[402], y_pred[403],
y_pred[404], y_pred[405], y_pred[406], y_pred[407], y_pred[408],
y_pred[409], y_pred[410], y_pred[411], y_pred[412], y_pred[413],
y_pred[414], y_pred[415], y_pred[416], y_pred[417], y_pred[418],
y_pred[419], y_pred[420], y_pred[421], y_pred[422], y_pred[423],
y_pred[424], y_pred[425], y_pred[426], y_pred[427], y_pred[428],
y_pred[429], y_pred[430], y_pred[431], y_pred[432], y_pred[433],
y_pred[434], y_pred[435], y_pred[436], y_pred[437], y_pred[438],
y_pred[439], y_pred[440], y_pred[441], y_pred[442], y_pred[443],
y_pred[444], y_pred[445], y_pred[446], y_pred[447], y_pred[448],
y_pred[449], y_pred[450], y_pred[451], y_pred[452], y_pred[453],
y_pred[454], y_pred[455], y_pred[456], y_pred[457], y_pred[458],
y_pred[459], y_pred[460], y_pred[461], y_pred[462], y_pred[463],
y_pred[464], y_pred[465], y_pred[466], y_pred[467], y_pred[468],
y_pred[469], y_pred[470], y_pred[471], y_pred[472], y_pred[473],
y_pred[474], y_pred[475], y_pred[476], y_pred[477], y_pred[478],
y_pred[479], y_pred[480], y_pred[481], y_pred[482], y_pred[483],
y_pred[484], y_pred[485], y_pred[486], y_pred[487], y_pred[488],
y_pred[489], y_pred[490], y_pred[491], y_pred[492], y_pred[493],
y_pred[494], y_pred[495], y_pred[496], y_pred[497], y_pred[498],
y_pred[499], y_pred[500], y_pred[501], y_pred[502], y_pred[503],
y_pred[504], y_pred[505], y_pred[506], y_pred[507], y_pred[508],
y_pred[509], y_pred[510], y_pred[511], y_pred[512], y_pred[513],
y_pred[514], y_pred[515], y_pred[516], y_pred[517], y_pred[518],
y_pred[519], y_pred[520], y_pred[521], y_pred[522], y_pred[523],
y_pred[524], y_pred[525], y_pred[526], y_pred[527], y_pred[528],
y_pred[529], y_pred[530], y_pred[531], y_pred[532], y_pred[533],
y_pred[534], y_pred[535], y_pred[536], y_pred[537], y_pred[538],
y_pred[539], y_pred[540], y_pred[541], y_pred[542], y_pred[543],
y_pred[544], y_pred[545], y_pred[546], y_pred[547], y_pred[548],
y_pred[549], y_pred[550], y_pred[551], y_pred[552], y_pred[553],

y_pred[554], y_pred[555], y_pred[556], y_pred[557], y_pred[558],
y_pred[559], y_pred[560], y_pred[561], y_pred[562], y_pred[563],
y_pred[564], y_pred[565], y_pred[566], y_pred[567], y_pred[568],
y_pred[569], y_pred[570], y_pred[571], y_pred[572], y_pred[573],
y_pred[574], y_pred[575], y_pred[576], y_pred[577], y_pred[578],
y_pred[579], y_pred[580], y_pred[581], y_pred[582], y_pred[583],
y_pred[584], y_pred[585], y_pred[586], y_pred[587], y_pred[588],
y_pred[589], y_pred[590], y_pred[591], y_pred[592], y_pred[593],
y_pred[594], y_pred[595], y_pred[596], y_pred[597], y_pred[598],
y_pred[599], y_pred[600], y_pred[601], y_pred[602], y_pred[603],
y_pred[604], y_pred[605], y_pred[606], y_pred[607], y_pred[608],
y_pred[609], y_pred[610], y_pred[611], y_pred[612], y_pred[613],
y_pred[614], y_pred[615], y_pred[616], y_pred[617], y_pred[618],
y_pred[619], y_pred[620], y_pred[621], y_pred[622], y_pred[623],
y_pred[624], y_pred[625], y_pred[626], y_pred[627], y_pred[628],
y_pred[629], y_pred[630], y_pred[631], y_pred[632], y_pred[633],
y_pred[634], y_pred[635], y_pred[636], y_pred[637], y_pred[638],
y_pred[639], y_pred[640], y_pred[641], y_pred[642], y_pred[643],
y_pred[644], y_pred[645], y_pred[646], y_pred[647], y_pred[648],
y_pred[649], y_pred[650], y_pred[651], y_pred[652], y_pred[653],
y_pred[654], y_pred[655], y_pred[656], y_pred[657], y_pred[658],
y_pred[659], y_pred[660], y_pred[661], y_pred[662], y_pred[663],
y_pred[664], y_pred[665], y_pred[666], y_pred[667], y_pred[668],
y_pred[669], y_pred[670], y_pred[671], y_pred[672], y_pred[673],
y_pred[674], y_pred[675], y_pred[676], y_pred[677], y_pred[678],
y_pred[679], y_pred[680], y_pred[681], y_pred[682], y_pred[683],
y_pred[684], y_pred[685], y_pred[686], y_pred[687], y_pred[688],
y_pred[689], y_pred[690], y_pred[691], y_pred[692], y_pred[693],
y_pred[694], y_pred[695], y_pred[696], y_pred[697], y_pred[698],
y_pred[699], y_pred[700], y_pred[701], y_pred[702], y_pred[703],
y_pred[704], y_pred[705], y_pred[706], y_pred[707], y_pred[708],
y_pred[709], y_pred[710], y_pred[711], y_pred[712], y_pred[713],
y_pred[714], y_pred[715], y_pred[716], y_pred[717], y_pred[718],
y_pred[719], y_pred[720], y_pred[721], y_pred[722], y_pred[723],
y_pred[724], y_pred[725], y_pred[726], y_pred[727], y_pred[728],
y_pred[729], y_pred[730], y_pred[731], y_pred[732], y_pred[733],
y_pred[734], y_pred[735], y_pred[736], y_pred[737], y_pred[738],
y_pred[739], y_pred[740], y_pred[741], y_pred[742], y_pred[743],
y_pred[744], y_pred[745], y_pred[746], y_pred[747], y_pred[748],
y_pred[749], y_pred[750], y_pred[751], y_pred[752], y_pred[753],
y_pred[754], y_pred[755], y_pred[756], y_pred[757], y_pred[758],
y_pred[759], y_pred[760], y_pred[761], y_pred[762], y_pred[763],
y_pred[764], y_pred[765], y_pred[766], y_pred[767], y_pred[768],

y_pred[769], y_pred[770], y_pred[771], y_pred[772], y_pred[773],
y_pred[774], y_pred[775], y_pred[776], y_pred[777], y_pred[778],
y_pred[779], y_pred[780], y_pred[781], y_pred[782], y_pred[783],
y_pred[784], y_pred[785], y_pred[786], y_pred[787], y_pred[788],
y_pred[789], y_pred[790], y_pred[791], y_pred[792], y_pred[793],
y_pred[794], y_pred[795], y_pred[796], y_pred[797], y_pred[798],
y_pred[799], y_pred[800], y_pred[801], y_pred[802], y_pred[803],
y_pred[804], y_pred[805], y_pred[806], y_pred[807], y_pred[808],
y_pred[809], y_pred[810], y_pred[811], y_pred[812], y_pred[813],
y_pred[814], y_pred[815], y_pred[816], y_pred[817], y_pred[818],
y_pred[819], y_pred[820], y_pred[821], y_pred[822], y_pred[823],
y_pred[824], y_pred[825], y_pred[826], y_pred[827], y_pred[828],
y_pred[829], y_pred[830], y_pred[831], y_pred[832], y_pred[833],
y_pred[834], y_pred[835], y_pred[836], y_pred[837], y_pred[838],
y_pred[839], y_pred[840], y_pred[841], y_pred[842], y_pred[843],
y_pred[844], y_pred[845], y_pred[846], y_pred[847], y_pred[848],
y_pred[849], y_pred[850], y_pred[851], y_pred[852], y_pred[853],
y_pred[854], y_pred[855], y_pred[856], y_pred[857], y_pred[858],
y_pred[859], y_pred[860], y_pred[861], y_pred[862], y_pred[863],
y_pred[864], y_pred[865], y_pred[866], y_pred[867], y_pred[868],
y_pred[869], y_pred[870], y_pred[871], y_pred[872], y_pred[873],
y_pred[874], y_pred[875], y_pred[876], y_pred[877], y_pred[878],
y_pred[879], y_pred[880], y_pred[881], y_pred[882], y_pred[883],
y_pred[884], y_pred[885], y_pred[886], y_pred[887], y_pred[888],
y_pred[889], y_pred[890], y_pred[891], y_pred[892], y_pred[893],
y_pred[894], y_pred[895], y_pred[896], y_pred[897], y_pred[898],
y_pred[899], y_pred[900], y_pred[901], y_pred[902], y_pred[903],
y_pred[904], y_pred[905], y_pred[906], y_pred[907], y_pred[908],
y_pred[909], y_pred[910], y_pred[911], y_pred[912], y_pred[913],
y_pred[914], y_pred[915], y_pred[916], y_pred[917], y_pred[918],
y_pred[919], y_pred[920], y_pred[921], y_pred[922], y_pred[923],
y_pred[924], y_pred[925], y_pred[926], y_pred[927], y_pred[928],
y_pred[929], y_pred[930], y_pred[931], y_pred[932], y_pred[933],
y_pred[934], y_pred[935], y_pred[936], y_pred[937], y_pred[938],
y_pred[939], y_pred[940], y_pred[941], y_pred[942], y_pred[943],
y_pred[944], y_pred[945], y_pred[946], y_pred[947], y_pred[948],
y_pred[949], y_pred[950], y_pred[951], y_pred[952], y_pred[953],
y_pred[954], y_pred[955], y_pred[956], y_pred[957], y_pred[958],
y_pred[959], y_pred[960], y_pred[961], y_pred[962], y_pred[963],
y_pred[964], y_pred[965], y_pred[966], y_pred[967], y_pred[968],
y_pred[969], y_pred[970], y_pred[971], y_pred[972], y_pred[973],
y_pred[974], y_pred[975], y_pred[976], y_pred[977], y_pred[978],
y_pred[979], y_pred[980], y_pred[981], y_pred[982], y_pred[983],

y_pred[984], y_pred[985], y_pred[986], y_pred[987], y_pred[988],
y_pred[989], y_pred[990], y_pred[991], y_pred[992], y_pred[993],
y_pred[994], y_pred[995], y_pred[996], y_pred[997], y_pred[998],
y_pred[999], y_pred[1000] triggered diagnostic warnings.

The expectands alpha, beta[1], beta[2], beta[3], p[1], p[2], p[3],
p[4], p[5], p[6], p[7], p[8], p[9], p[10], p[11], p[12], p[13], p[14],
p[15], p[16], p[17], p[18], p[19], p[20], p[21], p[22], p[23], p[24],
p[25], p[26], p[27], p[28], p[29], p[30], p[31], p[32], p[33], p[34],
p[35], p[37], p[38], p[39], p[40], p[41], p[42], p[43], p[44], p[45],
p[46], p[47], p[48], p[49], p[50], p[51], p[52], p[53], p[54], p[55],
p[56], p[57], p[58], p[59], p[60], p[61], p[62], p[63], p[64], p[65],
p[66], p[67], p[68], p[69], p[70], p[71], p[72], p[73], p[74], p[75],
p[76], p[77], p[78], p[79], p[80], p[81], p[82], p[83], p[84], p[85],
p[86], p[87], p[88], p[89], p[90], p[91], p[92], p[93], p[94], p[95],
p[96], p[97], p[98], p[99], p[100], p[101], p[102], p[103], p[104],
p[105], p[106], p[107], p[108], p[109], p[111], p[112], p[113], p[114],
p[115], p[116], p[118], p[120], p[121], p[122], p[123], p[124], p[125],
p[126], p[127], p[128], p[129], p[130], p[131], p[132], p[133], p[134],
p[135], p[136], p[137], p[138], p[139], p[140], p[141], p[142], p[143],
p[144], p[145], p[146], p[147], p[148], p[149], p[150], p[151], p[152],
p[153], p[154], p[155], p[156], p[157], p[158], p[159], p[160], p[161],
p[162], p[163], p[164], p[165], p[166], p[167], p[168], p[169], p[170],
p[171], p[172], p[173], p[174], p[175], p[176], p[177], p[179], p[180],
p[181], p[182], p[183], p[184], p[185], p[186], p[187], p[188], p[189],
p[190], p[191], p[192], p[193], p[194], p[195], p[196], p[197], p[198],
p[199], p[200], p[201], p[202], p[203], p[204], p[205], p[206], p[207],
p[208], p[209], p[210], p[211], p[212], p[213], p[214], p[215], p[216],
p[217], p[218], p[219], p[220], p[222], p[223], p[224], p[225], p[226],
p[227], p[228], p[229], p[230], p[231], p[232], p[233], p[234], p[235],
p[236], p[237], p[238], p[239], p[240], p[241], p[242], p[243], p[244],
p[245], p[246], p[248], p[249], p[250], p[251], p[252], p[253], p[254],
p[255], p[256], p[257], p[258], p[259], p[260], p[261], p[263], p[264],
p[265], p[266], p[267], p[268], p[269], p[270], p[271], p[272], p[273],
p[274], p[275], p[276], p[277], p[278], p[279], p[280], p[281], p[282],
p[283], p[284], p[285], p[286], p[287], p[288], p[289], p[290], p[291],
p[292], p[293], p[294], p[295], p[296], p[297], p[298], p[299], p[300],
p[301], p[302], p[303], p[304], p[305], p[306], p[307], p[308], p[309],
p[310], p[311], p[312], p[313], p[314], p[315], p[316], p[317], p[318],
p[319], p[320], p[321], p[322], p[323], p[324], p[325], p[326], p[327],
p[328], p[329], p[330], p[331], p[332], p[333], p[334], p[335], p[336],
p[337], p[338], p[339], p[340], p[341], p[342], p[343], p[344], p[345],
p[346], p[347], p[348], p[349], p[350], p[351], p[352], p[353], p[354],

p[356], p[357], p[358], p[359], p[360], p[361], p[362], p[363], p[364],
p[365], p[366], p[367], p[368], p[369], p[370], p[371], p[372], p[373],
p[374], p[375], p[376], p[377], p[378], p[379], p[380], p[381], p[382],
p[383], p[384], p[385], p[386], p[387], p[388], p[389], p[390], p[391],
p[392], p[393], p[394], p[395], p[396], p[397], p[398], p[399], p[400],
p[401], p[402], p[403], p[404], p[405], p[407], p[408], p[409], p[410],
p[411], p[412], p[413], p[414], p[415], p[416], p[417], p[418], p[419],
p[420], p[421], p[422], p[423], p[424], p[425], p[426], p[427], p[428],
p[429], p[430], p[431], p[432], p[433], p[434], p[435], p[436], p[437],
p[438], p[439], p[440], p[441], p[443], p[444], p[445], p[446], p[447],
p[448], p[449], p[450], p[451], p[452], p[453], p[454], p[455], p[456],
p[457], p[458], p[459], p[460], p[461], p[462], p[463], p[464], p[465],
p[466], p[467], p[468], p[469], p[470], p[471], p[472], p[473], p[474],
p[475], p[476], p[477], p[478], p[479], p[480], p[481], p[482], p[483],
p[484], p[485], p[486], p[487], p[488], p[489], p[490], p[491], p[492],
p[493], p[494], p[495], p[496], p[497], p[498], p[499], p[500], p[501],
p[502], p[503], p[504], p[505], p[506], p[507], p[508], p[509], p[510],
p[511], p[512], p[513], p[514], p[515], p[516], p[517], p[518], p[519],
p[520], p[521], p[522], p[523], p[524], p[525], p[526], p[527], p[528],
p[529], p[530], p[531], p[532], p[533], p[534], p[535], p[536], p[537],
p[539], p[540], p[541], p[542], p[543], p[544], p[545], p[546], p[547],
p[548], p[549], p[550], p[551], p[552], p[553], p[554], p[555], p[556],
p[557], p[558], p[559], p[561], p[562], p[563], p[564], p[565], p[566],
p[567], p[568], p[569], p[570], p[571], p[572], p[573], p[574], p[575],
p[576], p[577], p[578], p[579], p[580], p[581], p[582], p[583], p[584],
p[585], p[586], p[587], p[588], p[589], p[590], p[591], p[592], p[593],
p[594], p[595], p[596], p[597], p[598], p[599], p[600], p[601], p[602],
p[604], p[605], p[606], p[607], p[608], p[609], p[610], p[611], p[612],
p[613], p[614], p[615], p[616], p[617], p[618], p[619], p[620], p[621],
p[622], p[623], p[624], p[625], p[626], p[627], p[628], p[629], p[630],
p[631], p[632], p[633], p[634], p[635], p[636], p[637], p[638], p[639],
p[640], p[641], p[642], p[643], p[644], p[645], p[646], p[647], p[648],
p[649], p[650], p[651], p[652], p[653], p[654], p[655], p[656], p[657],
p[658], p[659], p[660], p[661], p[662], p[663], p[664], p[665], p[666],
p[667], p[668], p[669], p[670], p[671], p[672], p[673], p[674], p[675],
p[676], p[677], p[678], p[679], p[680], p[681], p[682], p[683], p[684],
p[685], p[686], p[687], p[688], p[689], p[690], p[691], p[692], p[693],
p[694], p[695], p[696], p[697], p[698], p[699], p[700], p[701], p[702],
p[703], p[704], p[705], p[706], p[707], p[708], p[709], p[710], p[711],
p[712], p[713], p[714], p[715], p[716], p[717], p[718], p[719], p[720],
p[721], p[722], p[723], p[724], p[725], p[726], p[727], p[728], p[729],
p[730], p[731], p[733], p[734], p[735], p[736], p[737], p[738], p[739],
p[740], p[741], p[742], p[743], p[744], p[745], p[746], p[747], p[748],

p[749], p[750], p[751], p[752], p[753], p[754], p[755], p[756], p[757],
 p[758], p[759], p[760], p[761], p[762], p[763], p[764], p[765], p[766],
 p[767], p[768], p[769], p[770], p[771], p[772], p[773], p[774], p[775],
 p[776], p[777], p[778], p[779], p[780], p[781], p[782], p[783], p[784],
 p[785], p[786], p[787], p[788], p[789], p[790], p[791], p[792], p[793],
 p[794], p[795], p[796], p[797], p[798], p[799], p[801], p[802], p[803],
 p[804], p[805], p[806], p[807], p[808], p[809], p[810], p[811], p[812],
 p[813], p[814], p[815], p[816], p[817], p[818], p[819], p[820], p[821],
 p[822], p[823], p[824], p[825], p[826], p[827], p[828], p[829], p[830],
 p[831], p[832], p[833], p[834], p[835], p[836], p[837], p[838], p[839],
 p[840], p[841], p[842], p[843], p[844], p[845], p[846], p[847], p[848],
 p[849], p[850], p[852], p[853], p[854], p[855], p[856], p[857], p[858],
 p[859], p[860], p[861], p[862], p[863], p[864], p[865], p[866], p[867],
 p[868], p[870], p[871], p[872], p[873], p[874], p[875], p[876], p[877],
 p[878], p[879], p[880], p[881], p[882], p[883], p[884], p[885], p[886],
 p[887], p[888], p[889], p[890], p[891], p[892], p[893], p[894], p[895],
 p[896], p[897], p[898], p[899], p[900], p[901], p[902], p[904], p[905],
 p[906], p[907], p[908], p[909], p[910], p[911], p[912], p[913], p[914],
 p[915], p[916], p[917], p[918], p[919], p[920], p[921], p[922], p[923],
 p[924], p[925], p[926], p[927], p[928], p[929], p[930], p[931], p[932],
 p[933], p[934], p[935], p[936], p[937], p[938], p[939], p[940], p[941],
 p[942], p[943], p[944], p[945], p[946], p[947], p[948], p[949], p[950],
 p[951], p[952], p[953], p[954], p[955], p[956], p[957], p[958], p[959],
 p[960], p[961], p[962], p[963], p[964], p[965], p[966], p[967], p[968],
 p[969], p[970], p[971], p[972], p[973], p[974], p[975], p[976], p[977],
 p[978], p[979], p[980], p[981], p[982], p[983], p[984], p[985], p[986],
 p[987], p[988], p[989], p[991], p[992], p[993], p[994], p[995], p[996],
 p[997], p[998], p[999], p[1000] triggered hat{R} warnings.

Split Rhat larger than 1.1 suggests that at least one of the Markov
 chains has not reached an equilibrium.

The expectands alpha, beta[1], beta[2], beta[3], p[1], p[2], p[3],
 p[4], p[5], p[6], p[7], p[8], p[9], p[10], p[11], p[12], p[13], p[14],
 p[15], p[16], p[17], p[18], p[19], p[20], p[21], p[22], p[23], p[24],
 p[25], p[26], p[27], p[28], p[29], p[30], p[31], p[32], p[33], p[34],
 p[35], p[36], p[37], p[38], p[39], p[40], p[41], p[42], p[43], p[44],
 p[45], p[46], p[47], p[48], p[49], p[50], p[51], p[52], p[53], p[54],
 p[55], p[56], p[57], p[58], p[59], p[60], p[61], p[62], p[63], p[64],
 p[65], p[66], p[67], p[68], p[69], p[70], p[71], p[72], p[73], p[74],
 p[75], p[76], p[77], p[78], p[79], p[80], p[81], p[82], p[83], p[84],
 p[85], p[86], p[87], p[88], p[89], p[90], p[91], p[92], p[93], p[94],
 p[95], p[96], p[97], p[98], p[99], p[100], p[101], p[102], p[103],

p[104], p[105], p[106], p[107], p[108], p[109], p[110], p[111], p[112],
p[113], p[114], p[115], p[116], p[117], p[118], p[119], p[120], p[121],
p[122], p[123], p[124], p[125], p[126], p[127], p[128], p[129], p[130],
p[131], p[132], p[133], p[134], p[135], p[136], p[137], p[138], p[139],
p[140], p[141], p[142], p[143], p[144], p[145], p[146], p[147], p[148],
p[149], p[150], p[151], p[152], p[153], p[154], p[155], p[156], p[157],
p[158], p[159], p[160], p[161], p[162], p[163], p[164], p[165], p[166],
p[167], p[168], p[169], p[170], p[171], p[172], p[173], p[174], p[175],
p[176], p[177], p[178], p[179], p[180], p[181], p[182], p[183], p[184],
p[185], p[186], p[187], p[188], p[189], p[190], p[191], p[192], p[193],
p[194], p[195], p[196], p[197], p[198], p[199], p[200], p[201], p[202],
p[203], p[204], p[205], p[206], p[207], p[208], p[209], p[210], p[211],
p[212], p[213], p[214], p[215], p[216], p[217], p[218], p[219], p[220],
p[221], p[222], p[223], p[224], p[225], p[226], p[227], p[228], p[229],
p[230], p[231], p[232], p[233], p[234], p[235], p[236], p[237], p[238],
p[239], p[240], p[241], p[242], p[243], p[244], p[245], p[246], p[247],
p[248], p[249], p[250], p[251], p[252], p[253], p[254], p[255], p[256],
p[257], p[258], p[259], p[260], p[261], p[262], p[263], p[264], p[265],
p[266], p[267], p[268], p[269], p[270], p[271], p[272], p[273], p[274],
p[275], p[276], p[277], p[278], p[279], p[280], p[281], p[282], p[283],
p[284], p[285], p[286], p[287], p[288], p[289], p[290], p[291], p[292],
p[293], p[294], p[295], p[296], p[297], p[298], p[299], p[300], p[301],
p[302], p[303], p[304], p[305], p[306], p[307], p[308], p[309], p[310],
p[311], p[312], p[313], p[314], p[315], p[316], p[317], p[318], p[319],
p[320], p[321], p[322], p[323], p[324], p[325], p[326], p[327], p[328],
p[329], p[330], p[331], p[332], p[333], p[334], p[335], p[336], p[337],
p[338], p[339], p[340], p[341], p[342], p[343], p[344], p[345], p[346],
p[347], p[348], p[349], p[350], p[351], p[352], p[353], p[354], p[355],
p[356], p[357], p[358], p[359], p[360], p[361], p[362], p[363], p[364],
p[365], p[366], p[367], p[368], p[369], p[370], p[371], p[372], p[373],
p[374], p[375], p[376], p[377], p[378], p[379], p[380], p[381], p[382],
p[383], p[384], p[385], p[386], p[387], p[388], p[389], p[390], p[391],
p[392], p[393], p[394], p[395], p[396], p[397], p[398], p[399], p[400],
p[401], p[402], p[403], p[404], p[405], p[406], p[407], p[408], p[409],
p[410], p[411], p[412], p[413], p[414], p[415], p[416], p[417], p[418],
p[419], p[420], p[421], p[422], p[423], p[424], p[425], p[426], p[427],
p[428], p[429], p[430], p[431], p[432], p[433], p[434], p[435], p[436],
p[437], p[438], p[439], p[440], p[441], p[442], p[443], p[444], p[445],
p[446], p[447], p[448], p[449], p[450], p[451], p[452], p[453], p[454],
p[455], p[456], p[457], p[458], p[459], p[460], p[461], p[462], p[463],
p[464], p[465], p[466], p[467], p[468], p[469], p[470], p[471], p[472],
p[473], p[474], p[475], p[476], p[477], p[478], p[479], p[480], p[481],
p[482], p[483], p[484], p[485], p[486], p[487], p[488], p[489], p[490],

p[491], p[492], p[493], p[494], p[495], p[496], p[497], p[498], p[499],
p[500], p[501], p[502], p[503], p[504], p[505], p[506], p[507], p[508],
p[509], p[510], p[511], p[512], p[513], p[514], p[515], p[516], p[517],
p[518], p[519], p[520], p[521], p[522], p[523], p[524], p[525], p[526],
p[527], p[528], p[529], p[530], p[531], p[532], p[533], p[534], p[535],
p[536], p[537], p[538], p[539], p[540], p[541], p[542], p[543], p[544],
p[545], p[546], p[547], p[548], p[549], p[550], p[551], p[552], p[553],
p[554], p[555], p[556], p[557], p[558], p[559], p[560], p[561], p[562],
p[563], p[564], p[565], p[566], p[567], p[568], p[569], p[570], p[571],
p[572], p[573], p[574], p[575], p[576], p[577], p[578], p[579], p[580],
p[581], p[582], p[583], p[584], p[585], p[586], p[587], p[588], p[589],
p[590], p[591], p[592], p[593], p[594], p[595], p[596], p[597], p[598],
p[599], p[600], p[601], p[602], p[603], p[604], p[605], p[606], p[607],
p[608], p[609], p[610], p[611], p[612], p[613], p[614], p[615], p[616],
p[617], p[618], p[619], p[620], p[621], p[622], p[623], p[624], p[625],
p[626], p[627], p[628], p[629], p[630], p[631], p[632], p[633], p[634],
p[635], p[636], p[637], p[638], p[639], p[640], p[641], p[642], p[643],
p[644], p[645], p[646], p[647], p[648], p[649], p[650], p[651], p[652],
p[653], p[654], p[655], p[656], p[657], p[658], p[659], p[660], p[661],
p[662], p[663], p[664], p[665], p[666], p[667], p[668], p[669], p[670],
p[671], p[672], p[673], p[674], p[675], p[676], p[677], p[678], p[679],
p[680], p[681], p[682], p[683], p[684], p[685], p[686], p[687], p[688],
p[689], p[690], p[691], p[692], p[693], p[694], p[695], p[696], p[697],
p[698], p[699], p[700], p[701], p[702], p[703], p[704], p[705], p[706],
p[707], p[708], p[709], p[710], p[711], p[712], p[713], p[714], p[715],
p[716], p[717], p[718], p[719], p[720], p[721], p[722], p[723], p[724],
p[725], p[726], p[727], p[728], p[729], p[730], p[731], p[732], p[733],
p[734], p[735], p[736], p[737], p[738], p[739], p[740], p[741], p[742],
p[743], p[744], p[745], p[746], p[747], p[748], p[749], p[750], p[751],
p[752], p[753], p[754], p[755], p[756], p[757], p[758], p[759], p[760],
p[761], p[762], p[763], p[764], p[765], p[766], p[767], p[768], p[769],
p[770], p[771], p[772], p[773], p[774], p[775], p[776], p[777], p[778],
p[779], p[780], p[781], p[782], p[783], p[784], p[785], p[786], p[787],
p[788], p[789], p[790], p[791], p[792], p[793], p[794], p[795], p[796],
p[797], p[798], p[799], p[800], p[801], p[802], p[803], p[804], p[805],
p[806], p[807], p[808], p[809], p[810], p[811], p[812], p[813], p[814],
p[815], p[816], p[817], p[818], p[819], p[820], p[821], p[822], p[823],
p[824], p[825], p[826], p[827], p[828], p[829], p[830], p[831], p[832],
p[833], p[834], p[835], p[836], p[837], p[838], p[839], p[840], p[841],
p[842], p[843], p[844], p[845], p[846], p[847], p[848], p[849], p[850],
p[851], p[852], p[853], p[854], p[855], p[856], p[857], p[858], p[859],
p[860], p[861], p[862], p[863], p[864], p[865], p[866], p[867], p[868],
p[869], p[870], p[871], p[872], p[873], p[874], p[875], p[876], p[877],


```
p[878], p[879], p[880], p[881], p[882], p[883], p[884], p[885], p[886],
p[887], p[888], p[889], p[890], p[891], p[892], p[893], p[894], p[895],
p[896], p[897], p[898], p[899], p[900], p[901], p[902], p[903], p[904],
p[905], p[906], p[907], p[908], p[909], p[910], p[911], p[912], p[913],
p[914], p[915], p[916], p[917], p[918], p[919], p[920], p[921], p[922],
p[923], p[924], p[925], p[926], p[927], p[928], p[929], p[930], p[931],
p[932], p[933], p[934], p[935], p[936], p[937], p[938], p[939], p[940],
p[941], p[942], p[943], p[944], p[945], p[946], p[947], p[948], p[949],
p[950], p[951], p[952], p[953], p[954], p[955], p[956], p[957], p[958],
p[959], p[960], p[961], p[962], p[963], p[964], p[965], p[966], p[967],
p[968], p[969], p[970], p[971], p[972], p[973], p[974], p[975], p[976],
p[977], p[978], p[979], p[980], p[981], p[982], p[983], p[984], p[985],
p[986], p[987], p[988], p[989], p[990], p[991], p[992], p[993], p[994],
p[995], p[996], p[997], p[998], p[999], p[1000] triggered hat{ESS}
warnings.
```

Small empirical effective sample sizes result in imprecise Markov chain Monte Carlo estimators.

```
clip_output <- function(output, head, tail) {
  for(l in 1:head)
    cat(paste0(output[l], "\n"))
  cat("\n")
  cat(".....\n")
  cat(".....\n")
  cat(".....\n")
  cat("\n")
  N <- length(output)
  for(l in (N - tail):N)
    cat(paste0(output[l], "\n"))
}
```

```
clip_output(capture.output(util$check_all_expectand_diagnostics(samples)),
  27, 21)
```

alpha:

```
Split hat{R} (1.575) exceeds 1.1.
Chain 1: hat{ESS} (16.304) is smaller than desired (100).
Chain 2: hat{ESS} (16.147) is smaller than desired (100).
Chain 3: hat{ESS} (7.943) is smaller than desired (100).
Chain 4: hat{ESS} (6.556) is smaller than desired (100).
```

```

beta[1]:
  Split hat{R} (1.428) exceeds 1.1.
  Chain 1: hat{ESS} (6.385) is smaller than desired (100).
  Chain 2: hat{ESS} (10.338) is smaller than desired (100).
  Chain 3: hat{ESS} (11.444) is smaller than desired (100).
  Chain 4: hat{ESS} (12.705) is smaller than desired (100).

```

```

beta[2]:
  Split hat{R} (1.420) exceeds 1.1.
  Chain 1: hat{ESS} (12.499) is smaller than desired (100).
  Chain 2: hat{ESS} (26.170) is smaller than desired (100).
  Chain 3: hat{ESS} (5.394) is smaller than desired (100).
  Chain 4: hat{ESS} (5.913) is smaller than desired (100).

```

```

beta[3]:
  Split hat{R} (1.679) exceeds 1.1.
  Chain 1: hat{ESS} (7.130) is smaller than desired (100).
  Chain 2: hat{ESS} (5.677) is smaller than desired (100).
  Chain 3: hat{ESS} (5.136) is smaller than desired (100).
  Chain 4: hat{ESS} (13.012) is smaller than desired (100).

```

```

.....
.....
.....

```

```

y_pred[999]:
  Chain 1: Both left and right hat{xi}s are NaN.
  Chain 2: Both left and right hat{xi}s are NaN.
  Chain 3: Both left and right hat{xi}s are NaN.
  Chain 4: Both left and right hat{xi}s are NaN.

```

```

y_pred[1000]:
  Chain 1: Both left and right hat{xi}s are NaN.
  Chain 2: Both left and right hat{xi}s are NaN.
  Chain 3: Both left and right hat{xi}s are NaN.
  Chain 4: Both left and right hat{xi}s are NaN.

```

Large tail $\hat{x}_i$ s suggest that the expectand might not be sufficiently integrable.

Split R_{hat} larger than 1.1 suggests that at least one of the Markov

chains has not reached an equilibrium.

Small empirical effective sample sizes result in imprecise Markov chain Monte Carlo estimators.

Well that output restriction proved to be prescient as most of the expectands are encountering problems; even this compact summary is overwhelming. To avoid completely overwhelming ourselves let's focus on the four parameter expectands.

```
base_samples <- util$filter_expectands(samples,
                                       c('alpha', 'beta'),
                                       TRUE)
util$check_all_expectand_diagnostics(base_samples)
```

alpha:

```
Split hat{R} (1.575) exceeds 1.1.
Chain 1: hat{ESS} (16.304) is smaller than desired (100).
Chain 2: hat{ESS} (16.147) is smaller than desired (100).
Chain 3: hat{ESS} (7.943) is smaller than desired (100).
Chain 4: hat{ESS} (6.556) is smaller than desired (100).
```

beta[1]:

```
Split hat{R} (1.428) exceeds 1.1.
Chain 1: hat{ESS} (6.385) is smaller than desired (100).
Chain 2: hat{ESS} (10.338) is smaller than desired (100).
Chain 3: hat{ESS} (11.444) is smaller than desired (100).
Chain 4: hat{ESS} (12.705) is smaller than desired (100).
```

beta[2]:

```
Split hat{R} (1.420) exceeds 1.1.
Chain 1: hat{ESS} (12.499) is smaller than desired (100).
Chain 2: hat{ESS} (26.170) is smaller than desired (100).
Chain 3: hat{ESS} (5.394) is smaller than desired (100).
Chain 4: hat{ESS} (5.913) is smaller than desired (100).
```

beta[3]:

```
Split hat{R} (1.679) exceeds 1.1.
Chain 1: hat{ESS} (7.130) is smaller than desired (100).
Chain 2: hat{ESS} (5.677) is smaller than desired (100).
Chain 3: hat{ESS} (5.136) is smaller than desired (100).
Chain 4: hat{ESS} (13.012) is smaller than desired (100).
```

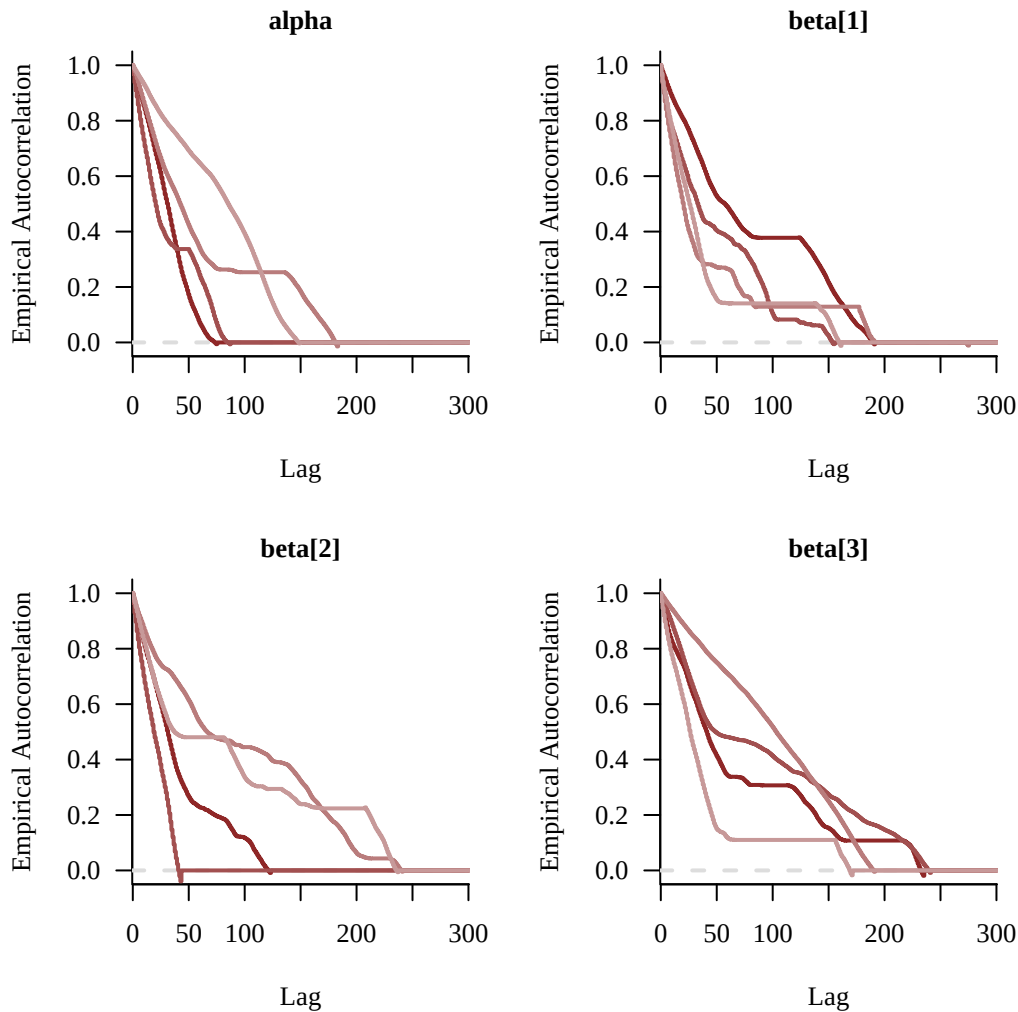
Split $\hat{R}$ larger than 1.1 suggests that at least one of the Markov chains has not reached an equilibrium.

Small empirical effective sample sizes result in imprecise Markov chain Monte Carlo estimators.

All four parameter expectands exhibit split $\hat{R}$ warnings and low empirical effective sample size warnings. The question is whether or not the split $\hat{R}$ warnings indicate quasistationarity or just insufficient exploration.

Motivated by the small effective sample size estimates let's look at the empirical correlograms for each parameter expectand.

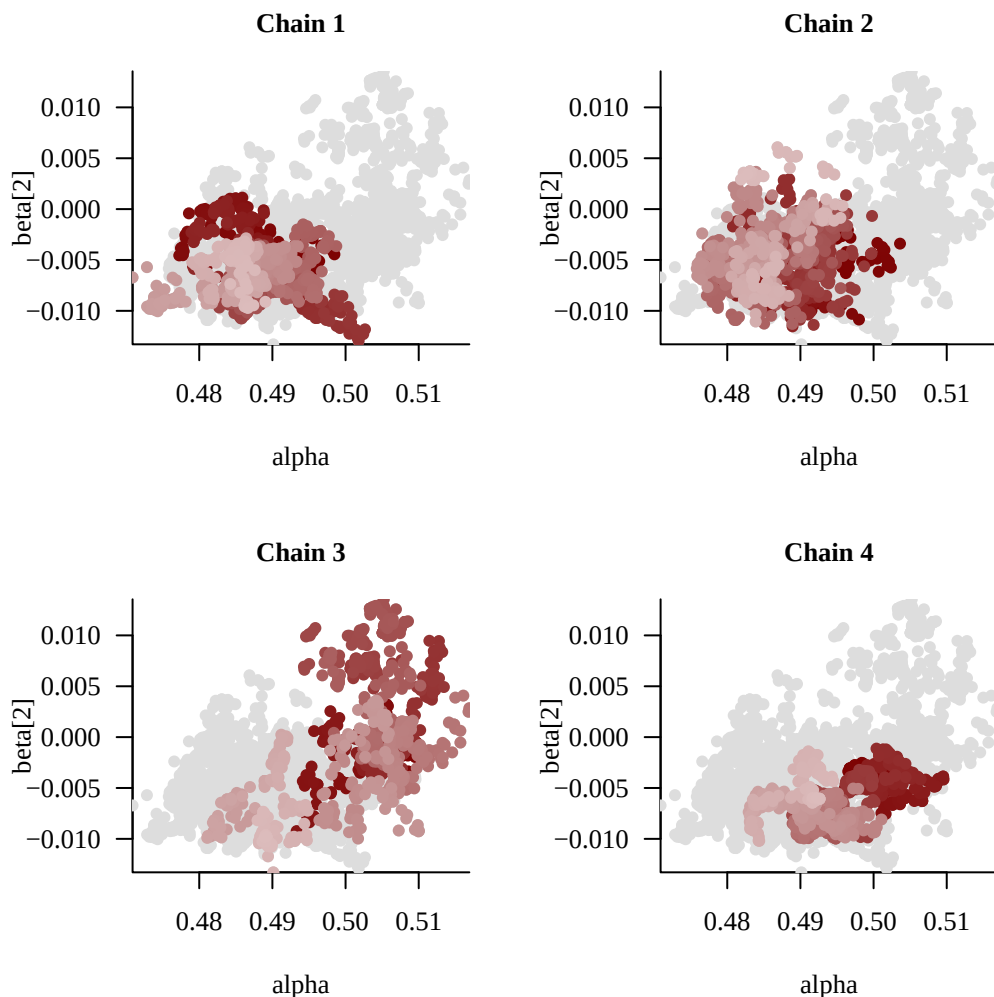
```
par(mfrow=c(2, 2), mar = c(5, 5, 2, 1))
util$plot_empirical_correlogram(samples[["alpha"]], 300,
                                rho_lim=c(-0.05, 1.05), "alpha")
util$plot_empirical_correlogram(samples[["beta[1]"]], 300,
                                rho_lim=c(-0.05, 1.05), "beta[1]")
util$plot_empirical_correlogram(samples[["beta[2]"]], 300,
                                rho_lim=c(-0.05, 1.05), "beta[2]")
util$plot_empirical_correlogram(samples[["beta[3]"]], 300,
                                rho_lim=c(-0.05, 1.05), "beta[3]")
```



Regardless of whether or not these Markov chains are stationary they are extremely autocorrelated. Assuming stationarity we don't start to forget the beginning of each Markov chain until we've worked through almost all of the total length, leaving the equivalent of only one independent sample across each chain.

This is consistent with the constraint violations breaking the coherent, gradient-driven exploration of Hamiltonian Monte Carlo so that the Markov chains devolve into diffuse random walks. Indeed looking at the chain-separated pairs plots we see the spatial color continuity characteristic of a random walk.

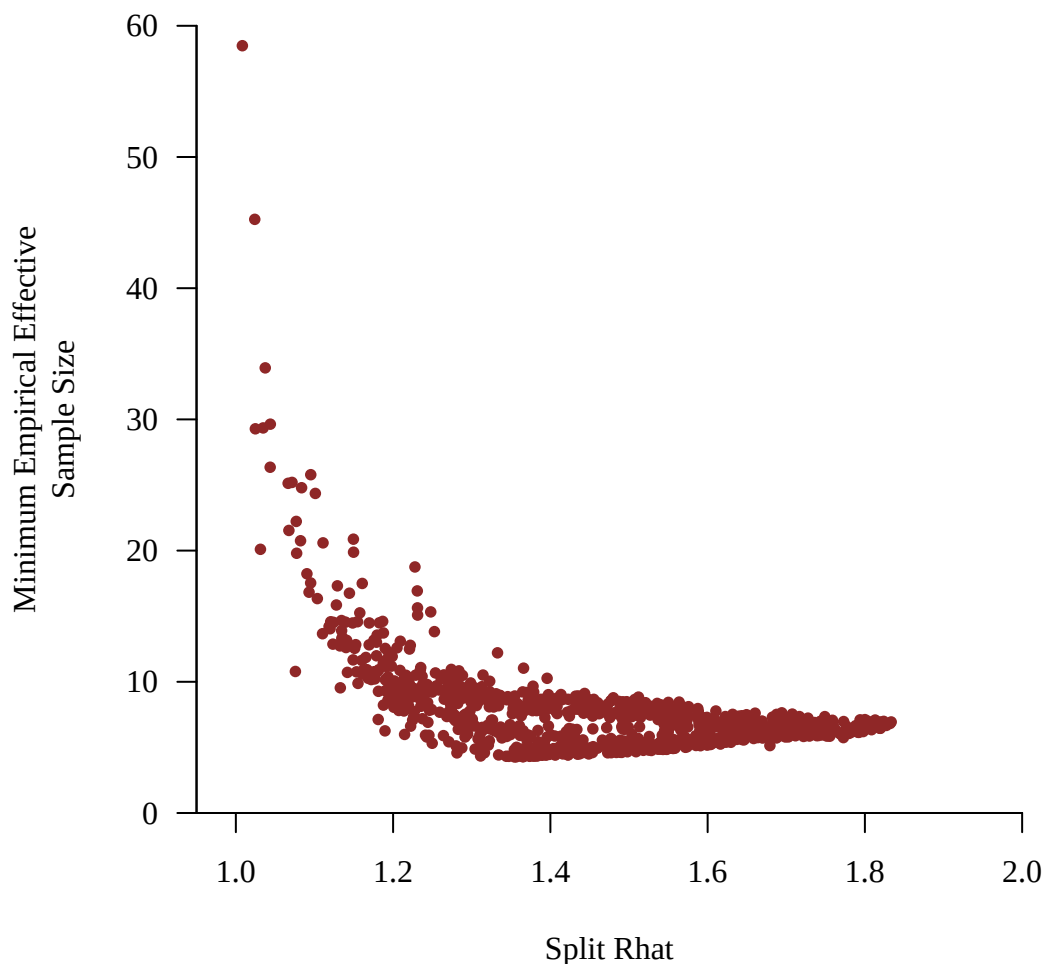
```
util$plot_pairs_by_chain(samples[["alpha"]], "alpha",
                        samples[["beta[2]"]], "beta[2]")
```



To more quantitatively blame the large split $\hat{R}$ s on these strong autocorrelations we can plot the split $\hat{R}$ from each expectand against the corresponding empirical effective sample size. Specifically for each expectand we plot split $\hat{R}$ against the smallest empirical effective sample size amongst the four Markov chains.

```
rhats <- util$compute_split_rhats(samples)
min_ess_hats <- util$compute_min_ess_hats(samples)

par(mfrow=c(1, 1), mar = c(5, 5, 2, 1))
plot(rhats, min_ess_hats,
     col=c_dark, pch=16, cex=0.8,
     xlab="Split Rhat", xlim=c(0.95, 2),
     ylab="Minimum Empirical Effective\nSample Size", ylim=c(0, 60))
```



Every expectand with a large split $\hat{R}_s$ also exhibits a particularly small minimum empirical effective sample size, confirming that the latter are likely due to our Markov chains not containing enough information.

If we are sloppy, ignore these diagnostics, and assume that all of our Markov chain Monte Carlo estimators are accurate then we are quickly misled about the actual behavior of the posterior distribution. One way to guard against this sloppiness is to always accompany a Markov chain Monte Carlo estimator with an estimated error. Even if that error is inaccurate it can sometimes communicate underlying problems.

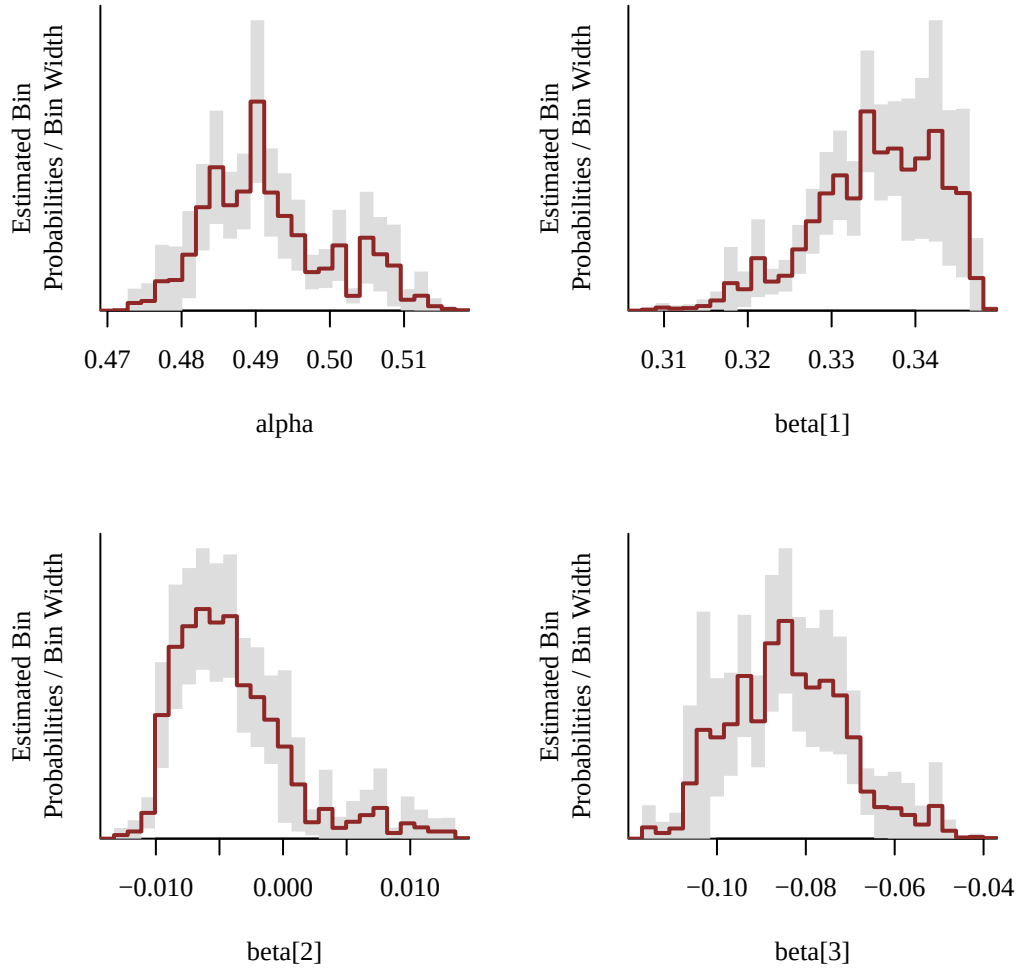
For example let's look at a pushforward histogram for each parameter with light gray bands visualizing the standard error around the bin probability estimates in dark red.

```
par(mfrow=c(2, 2), mar = c(5, 4, 2, 1))
util$plot_expectand_pushforward(samples[["alpha"]], 25,
                                display_name="alpha")
```

```

util$plot_expectand_pushforward(samples[["beta[1]"]], 25,
                                display_name="beta[1]")
util$plot_expectand_pushforward(samples[["beta[2]"]], 25,
                                display_name="beta[2]")
util$plot_expectand_pushforward(samples[["beta[3]"]], 25,
                                display_name="beta[3]")

```



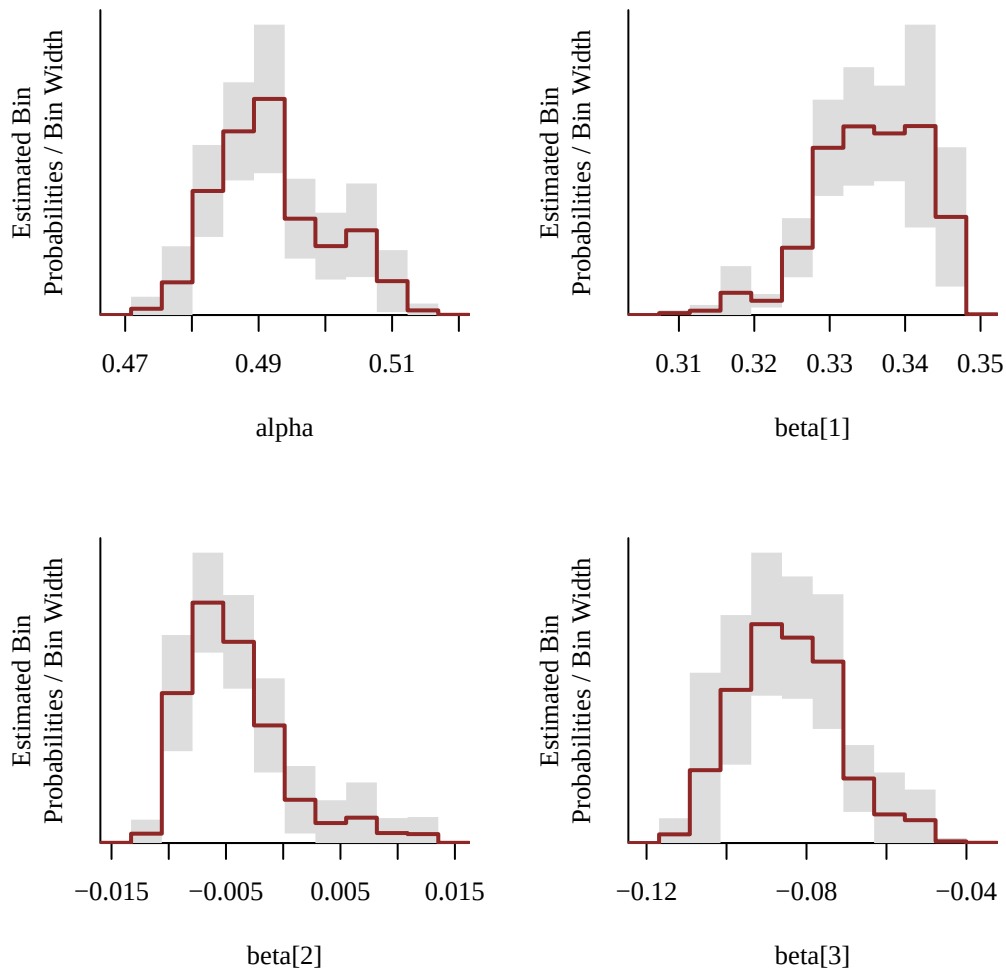
If we look at the central estimates alone we might convince ourselves of all kinds of interesting structure. For example potential multi-modality in **alpha** and **beta[2]** and platykurticity in **beta[1]** and **beta[3]**. These structures, however, are all within the scope of the relatively large standard error bands which suggests that they are all consistent with estimator noise.

Reducing the number of bins decreases the relative standard errors but at the same time many of the visual artifacts recede.


```

par(mfrow=c(2, 2), mar = c(5, 4, 2, 1))
util$plot_expectand_pushforward(samples[["alpha"]], 10,
                                display_name="alpha")
util$plot_expectand_pushforward(samples[["beta[1]"]], 10,
                                display_name="beta[1]")
util$plot_expectand_pushforward(samples[["beta[2]"]], 10,
                                display_name="beta[2]")
util$plot_expectand_pushforward(samples[["beta[3]"]], 10,
                                display_name="beta[3]")

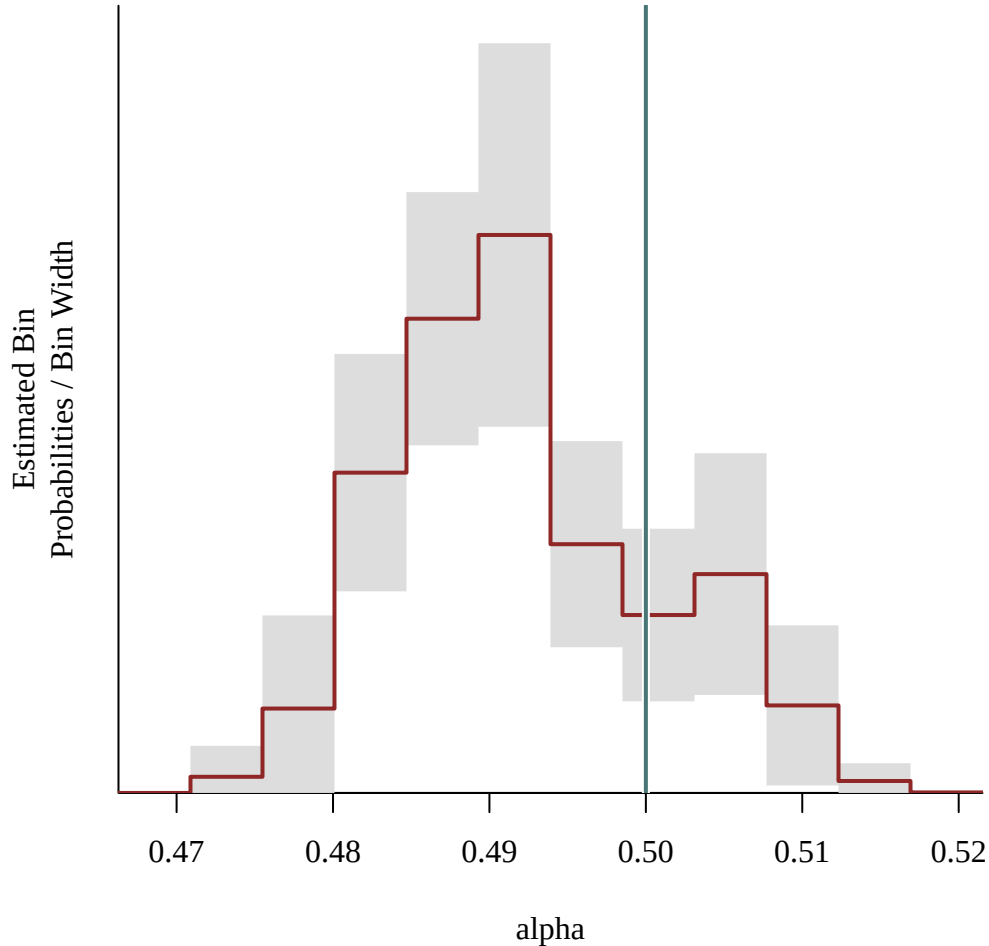
```



When the bin indicator functions enjoy Markov chain Monte Carlo central limit theorems these standard error bands allow us to discriminate between meaningful structure and accidental artifacts regardless of the histogram binning. Even if central limit theorems don't hold the error bands provide one more way that we can potentially diagnose untrustworthy computation.

The `plot_expectand_pushforward` can also overlay a baseline value for comparison, for example when comparing posterior inferences to the ground truth in simulation studies.

```
par(mfrow=c(1, 1), mar = c(5, 4, 2, 1))
util$plot_expectand_pushforward(samples[["alpha"]], 10,
                                display_name="alpha",
                                baseline=0.5,
                                baseline_col=util$c_mid_teal)
```



Moreover the expectand pushforward histograms can be plotted on top of each other for a more direct comparison.

```
par(mfrow=c(1, 1), mar=c(5, 5, 1, 1))
util$plot_expectand_pushforward(samples[["beta[1]"]], 50,
```

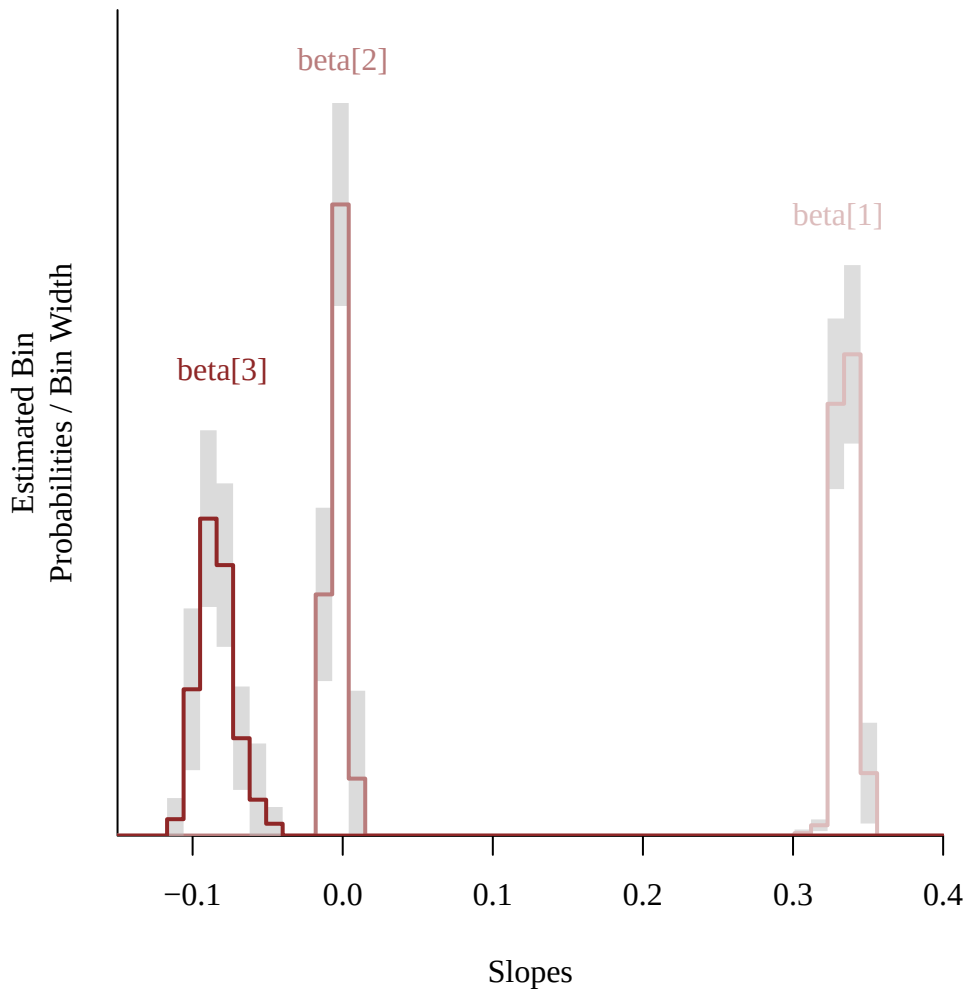
```

        flim=c(-0.15, 0.4),
        ylim=c(0, 80),
        display_name="Slopes",
        col=util$c_light)
text(0.33, 60, "beta[1]", col=util$c_light)

util$plot_expectand_pushforward(samples[["beta[2]"]], 50,
        flim=c(-0.15, 0.4),
        col=util$c_mid,
        border="#BBBBBB88",
        add=TRUE)
text(0.00, 75, "beta[2]", col=util$c_mid)

util$plot_expectand_pushforward(samples[["beta[3]"]], 50,
        flim=c(-0.15, 0.4),
        col=util$c_dark,
        border="#BBBBBB88",
        add=TRUE)
text(-0.08, 45, "beta[3]", col=util$c_dark)

```



Finally if we want to explore the pushforward posterior distribution of other expectands that have not already been evaluated in the `Stan` program then we need to evaluate them ourselves.

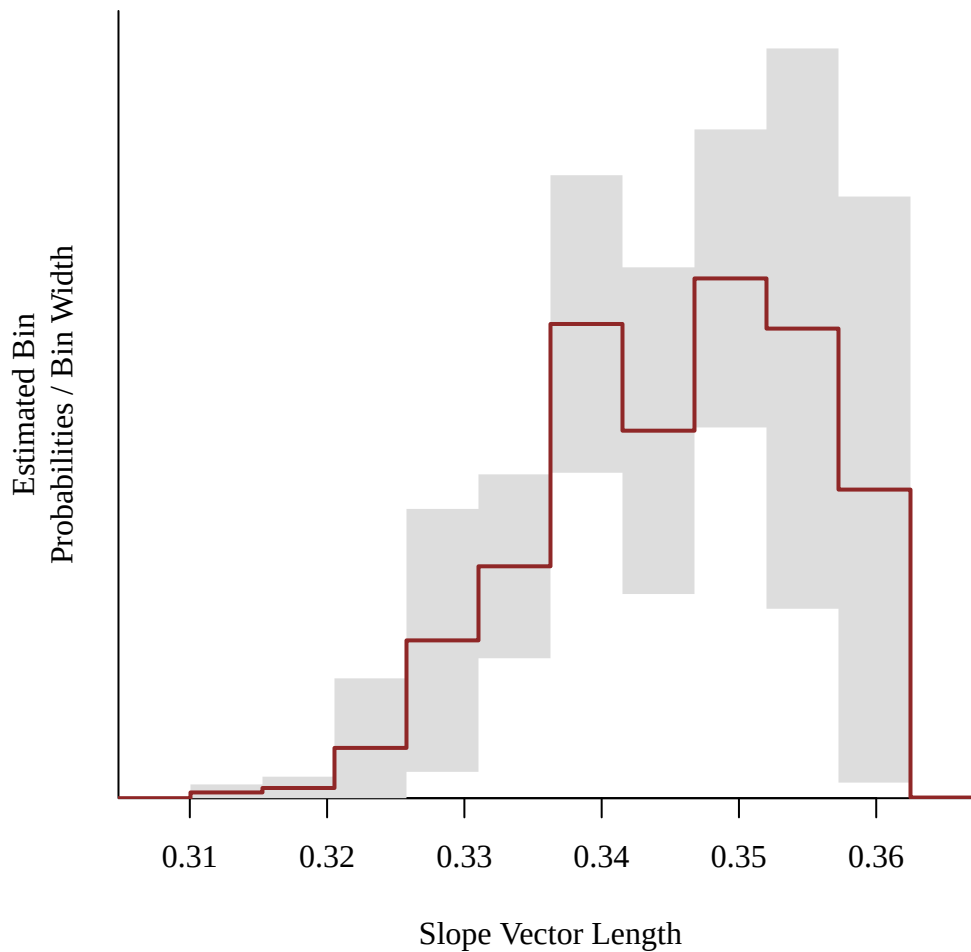
```
euclidean_length <- function(x) {
  sqrt(sum(x**2))
}

beta_names <- util$name_array('beta', c(data$M))

pushforward_samples <-
  util$eval_expectand_pushforward(samples, euclidean_length,
    list('x'=beta_names))

par(mfrow=c(1, 1), mar = c(5, 4, 2, 1))
```

```
util$plot_expectand_pushforward(pushforward_samples, 10,
                               display_name="Slope Vector Length")
```



Calculating the probability allocated to implicitly-defined subsets proceeds similarly.

```
pi_est <- util$implicit_subset_prob(samples,
                                   function(x) x > 0,
                                   list('x'='beta[2]'))

cat(paste0("Posterior probability that beta[2] is greater than zero = ",
           sprintf("%.2f +/- %.2f.", pi_est[1], 2 * pi_est[2])))
```

Posterior probability that beta[2] is greater than zero = 0.13 +/- 0.11.

License

The code in this case study is copyrighted by Michael Betancourt and licensed under the new BSD (3-clause) license:

<https://opensource.org/licenses/BSD-3-Clause>

The text and figures in this case study are copyrighted by Michael Betancourt and licensed under the CC BY-NC 4.0 license:

<https://creativecommons.org/licenses/by-nc/4.0/>

Original Computing Environment

```
writeLines(readLines(file.path(Sys.getenv("HOME"), ".R/Makevars")))
```

```
CC=clang
```

```
CXXFLAGS=-O3 -mtune=native -march=native -Wno-unused-variable -Wno-unused-function -Wno-macro-redefined  
CXX=clang++ -arch x86_64 -ftemplate-depth-256
```

```
CXX14FLAGS=-O3 -mtune=native -march=native -Wno-unused-variable -Wno-unused-function -Wno-macro-redefined  
CXX14=clang++ -arch x86_64 -ftemplate-depth-256
```

```
sessionInfo()
```

```
R version 4.3.2 (2023-10-31)  
Platform: x86_64-apple-darwin20 (64-bit)  
Running under: macOS Sonoma 14.7.5
```

```
Matrix products: default
```

```
BLAS: /Library/Frameworks/R.framework/Versions/4.3-x86_64/Resources/lib/libRblas.0.dylib  
LAPACK: /Library/Frameworks/R.framework/Versions/4.3-x86_64/Resources/lib/libRlapack.dylib;
```

```
locale:  
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/New_York  
tzcode source: internal
```

attached base packages:

[1] stats graphics grDevices utils datasets methods base

other attached packages:

[1] colormap_0.1.4 rstan_2.32.6 StanHeaders_2.32.7

loaded via a namespace (and not attached):

[1] gtable_0.3.4	jsonlite_1.8.8	compiler_4.3.2	Rcpp_1.0.11
[5] stringr_1.5.1	parallel_4.3.2	gridExtra_2.3	scales_1.3.0
[9] yaml_2.3.8	fastmap_1.1.1	ggplot2_3.4.4	R6_2.5.1
[13] curl_5.2.0	knitr_1.45	tibble_3.2.1	munSELL_0.5.0
[17] pillar_1.9.0	rlang_1.1.2	utf8_1.2.4	V8_4.4.1
[21] stringi_1.8.3	inline_0.3.19	xfun_0.41	RcppParallel_5.1.7
[25] cli_3.6.2	magrittr_2.0.3	digest_0.6.33	grid_4.3.2
[29] lifecycle_1.0.4	vctrs_0.6.5	evaluate_0.23	glue_1.6.2
[33] QuickJSR_1.0.8	codetools_0.2-19	stats4_4.3.2	pkgbuild_1.4.3
[37] fansi_1.0.6	colorspace_2.1-0	rmarkdown_2.25	matrixStats_1.2.0
[41] tools_4.3.2	loo_2.6.0	pkgconfig_2.0.3	htmltools_0.5.7

Stan

Program 1 simu_logistic_reg.stan

```
transformed data {
  int<lower=0> M = 3;          // Number of covariates
  int<lower=0> N = 1000;      // Number of observations

  vector[M] x0 = [-1, 0, 1]'; // Covariate baseline
  vector[M] z0 = [-3, 1, 2]'; // Latent functional behavior baseline
  real gamma0 = -2.6;         // True intercept
  vector[M] gamma1 = [0.2, -2.0, 0.33]'; // True slopes
  matrix[M, M] gamma2 = [ [+0.40, -0.05, -0.20],
                           [-0.05, -1.00, -0.05],
                           [-0.20, -0.05, +0.50] ];
}

generated quantities {
  matrix[N, M] X; // Covariate design matrix
  real y[N];      // Variates

  for (n in 1:N) {
    real x2 = -5;
    while (x2 < x0[2] - 4 || x2 > x0[2] + 4)
      x2 = normal_rng(x0[2], 2);

    X[n, 2] = x2;
    X[n, 1] = normal_rng(x0[1] + 1.0 * cos(1.5 * (X[n, 2] - x0[2])), 0.3);
    X[n, 3] = normal_rng(x0[3] + 0.76 * (X[n, 1] - x0[1]), 0.5);

    y[n] = bernoulli_logit_rng( gamma0
                                + (X[n] - z0') * gamma1
                                + (X[n] - z0') * gamma2 * (X[n] - z0')');
  }
}
```

Stan

Program 2 bernoulli_linear.stan

```
data {
  int<lower=0> M; // Number of covariates
  int<lower=0> N; // Number of observations

  vector[M] x0; // Covariate baselines
  matrix[N, M] X; // Covariate design matrix

  int<lower=0, upper=1> y[N]; // Variates
}

transformed data {
  matrix[N, M] deltaX;
  for (n in 1:N) {
    deltaX[n,] = X[n] - x0';
  }
}

parameters {
  real alpha; // Intercept
  vector[M] beta; // Linear slopes
}

model {
  // Prior model
  alpha ~ normal(0, 1);
  beta ~ normal(0, 1);

  // Vectorized observation model
  y ~ bernoulli(alpha + deltaX * beta);
}

// Simulate a full observation from the current value of the parameters
generated quantities {
  vector[N] p = alpha + deltaX * beta;
  int y_pred[N] = bernoulli_rng(p);
}
```